```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyRestaurant.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S14_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.CrazyRestaurant.Boot
{
    /// &lt;summary&gt;
```

```
/// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
/// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
    static bool ConfigStep()
    {
```

```
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyRestaurant.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
    }
    public sealed class MockTimeService : ITimeService
    {
```

```
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    // U184 real-device CJK face (B569 chain step 3, B657 wiring): LegacyRuntime.ttf
    // ships no zh glyphs, so the zh-CN data faces (Texts_zhCN.json lines) render
    // blank on real devices. The B569 fleet-cast NotoSansSC subset (201KB,
    // cmap-verified missing=0 over the fleet union incl. the CrazyRestaurant
    // 444-char census, FontGate PASS 444/444) rides Resources as
    // NotoSansSC-CrazySubset.otf; future view faces resolve this name and
    // degrade to LegacyRuntime on a miss.
    public FontSet GetFontSet()
        =&gt; new FontSet("NotoSansSC-CrazySubset.otf", "NotoSansSC-CrazySubset.otf", "NotoSansSC-CrazySubset.otf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
```

```
        public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
            =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
        public void ShowInterstitial(string pointId) { }
        public void ShowBanner(string pointId) { }
        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// ChefCards.cs
// ChefCards.cs - the quirky chef card system (G14-T006a, SYS03).
// Six hires on one closed roster (the frozen ChefTable). Draws are a pure
// rotation (zero RNG: the same pull count always yields the same card
// order), acquisitions ride the rank-gate face (the frozen chef rows carry
// no unlock rank - every row is met at any rank, the album progression
// gates itself through collection), and every completed album volume books
// the frozen +2% income bonus. The six passives resolve from the frozen
// closed set onto the serve engine: the cashier's mental math lifts dish
// prices, the zen prep runs half a beat slow (the comedy cost), the brave
// taster speeds the R&amp;D window, and the blaze chef arms the bangs-burn
// comedy (the future event spine consumes that face; the dishwasher and
// the speedster resolve on their own query faces until the dish and
// takeout spines land). The ten-pull rides the AdCap gacha category with
// the G08 T009 three-piece law: the HasCategoryBudget precheck and the
// consumption draw the same policy instance and the same day bucket (they
// can never disagree), the evt_ad funnel books offer -&gt; complete -&gt;
// landed in order, and a capped tap refuses BEFORE the ad ever plays.
using System;
using System.Collections.Generic;
using Sy.Platform;
namespace Sy.CrazyRestaurant
{
    /// &lt;summary&gt;One roster card: the compiled mirror of one frozen chef row
    /// (ids and passives come from RestaurantData, never restated here - a
    /// schema drift shows up as a check mismatch, not a copy).&lt;/summary&gt;
    public sealed class ChefCardDef
    {
        public string id = "";
        public int unlockRank;          // 0 for every chef row: the frozen table gates no rank
        public string passiveId = "";  // the frozen closed set
        public string albumId = "";
    }
    /// &lt;summary&gt;One ten-pull result: the rotation order, the first-time
    /// acquisitions, and the capped-refuse face (nothing played).&lt;/summary&gt;
    public sealed class ChefCardDraw
    {
        public string[] PulledIds = Array.Empty&lt;string&gt;();
        public string[] NewIds = Array.Empty&lt;string&gt;();
```

```
    public bool CappedRefused;
}
public sealed class ChefCards
{
    /// &lt;summary&gt;Ten-pull ad point (T006): draws from the 10/day
    /// g14_gacha category - one rewarded ad per ten-pull (AdCategories
    /// authority, g14_ prefix keeps id namespaces isolated).&lt;/summary&gt;
    public const string TenPullPointId = "g14_ten_pull";
    // ---- v1 engine wiring (sweep-tunable; frozen economy laws stay in
    // RestaurantData - these carry only the card feel) ----
    public const int TenPullSize = 10;            // ten draws per pull (the roster cycles - repeats are the dupes)
    public const int ZencutServePct = -10;        // zen prep: half a beat slow, the comedy cost
    public const int CashierPricePct = 10;        // the cashier girl: mental math beats the calculator
    public const int DishwasherWashPct = -15;      // the slacker dishwasher (query face until the dish spine lands)
    public const int SpeedyDeliveryPct = 20;      // the takeout speedster (query face until the takeout spine lands
    public const int TasterResearchPct = 15;       // the brave taster: the R&amp;amp;D window runs faster
    readonly RestaurantServe _serve;
    readonly AdCapPolicy _adCap;       // the shared policy instance (the three-piece law)
    readonly Func&lt;string&gt; _dayKey;     // injected for deterministic gates; production rides TodayKey
    readonly List&lt;string&gt; _owned = new List&lt;string&gt;();
    readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
    int _drawCursor;                    // zero RNG: the pull order is a fixed rotation
    int _tenPullsDone;
    public ChefCards(RestaurantServe serve, AdCapPolicy adCap, Func&lt;string&gt; dayKey = null)
    {
        _serve = serve;
        _adCap = adCap;
        _dayKey = dayKey ?? AdCapPolicy.TodayKey;
    }
    /// &lt;summary&gt;The closed roster: the six chefs, ids and passive rows
    /// read live from the frozen table.&lt;/summary&gt;
    public static ChefCardDef[] Roster()
    {
        var list = new List&lt;ChefCardDef&gt;();
        foreach (var c in RestaurantData.Chefs())
            list.Add(new ChefCardDef { id = c.id, unlockRank = 0, passiveId = c.passiveId, albumId = c.albumId });
        return list.ToArray();
    }
    // ---- live faces ----
    public RestaurantServe ServeOf =&gt; _serve;
    public IReadOnlyList&lt;string&gt; Owned =&gt; _owned;
    public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
    public int TenPullsDone =&gt; _tenPullsDone;
    public int DrawCursor =&gt; _drawCursor;
    public bool Owns(string cardId) =&gt; _owned.Contains(cardId);
    /// &lt;summary&gt;True when the rank has met the card's gate row (the
    /// frozen chef rows gate no rank - the face stays structural so the
    /// mirror never diverges from the FarmCards law).&lt;/summary&gt;
    public bool UnlockMet(ChefCardDef card)
        =&gt; card.unlockRank &lt;= _serve.Rank.rank;
```

```
// ---- the album faces ----
public static string[] AlbumVolumeIds()
    =&gt; new[] { "album_chef", "album_customer", "album_dark" };
public int VolumeSize(string albumId)
{
    int n = 0;
    foreach (var c in Roster()) if (c.albumId == albumId) n++;
    // the customer and dark volumes collect through their own spines
    // (reviews / dark research): their rows count toward the volume
    // size, none are drawable from the chef ten-pull.
    if (albumId == "album_customer") n = RestaurantData.CustomerCount;
    if (albumId == "album_dark") n = RestaurantData.DarkRecipeCount;
    return n;
}
public int OwnedInVolume(string albumId)
{
    int n = 0;
    foreach (var id in _owned) if (AlbumOf(id) == albumId) n++;
    return n;
}
public bool VolumeComplete(string albumId)
    =&gt; OwnedInVolume(albumId) &gt;= VolumeSize(albumId);
/// &lt;summary&gt;Completed volumes count (the +2% law rides
/// RestaurantData.AlbumBonusPerVolumePct).&lt;/summary&gt;
public int CompletedVolumes()
{
    int n = 0;
    foreach (var a in AlbumVolumeIds()) if (VolumeComplete(a)) n++;
    return n;
}
public int AlbumIncomeBonusPct =&gt; CompletedVolumes() * RestaurantData.AlbumBonusPerVolumePct;
static string AlbumOf(string cardId)
{
    foreach (var c in Roster()) if (c.id == cardId) return c.albumId;
    // T009a: the dark dishes map through their own frozen table -
    // a registered weird dish counts into the dark volume (the
    // rank-gate dish ridge and the +2% album law both ride it).
    foreach (var d in RestaurantData.DarkRecipes()) if (d.id == cardId) return d.albumId;
    return "";
}
// ---- the AdCap three-piece (G08 T009 law) ----
/// &lt;summary&gt;Live precheck: always-fresh budget face on the same policy
/// instance and the same day bucket the consumption draws from.&lt;/summary&gt;
public bool TenPullBudgetOk
    =&gt; _adCap != null &amp;&amp; _adCap.HasCategoryBudget(_dayKey(), AdCategories.CategoryOfPoint(TenPullPoin
/// &lt;summary&gt;The ten-pull: precheck first (a capped tap refuses BEFORE
/// the ad ever plays and books the capped funnel line), consume on the
/// same policy instance and day bucket, then resolve ten cards on the
/// zero-RNG rotation with the rank gate and book the funnel in order.
/// A null policy refuses nothing (deterministic gate face).&lt;/summary&gt;
```

```
public ChefCardDraw TryTenPull()
{
    var draw = new ChefCardDraw();
    if (_adCap != null)
    {
        if (!_adCap.HasCategoryBudget(_dayKey(), AdCategories.CategoryOfPoint(TenPullPointId)))
        {
            draw.CappedRefused = true;
            _funnel.Add("evt_ad_offer_capped@" + Stamp());
            return draw;   // the honest face refuses at the tap, never after the ad
        }
        if (!_adCap.TryConsumeRewarded(_dayKey(), AdCategories.CategoryOfPoint(TenPullPointId)))
        {
            draw.CappedRefused = true;
            _funnel.Add("evt_ad_offer_capped@" + Stamp());
            return draw;
        }
    }
    _tenPullsDone++;
    _funnel.Add("evt_ad_offer@" + Stamp());
    _funnel.Add("evt_ad_complete@" + Stamp());
    var roster = Roster();
    var pulled = new List&lt;string&gt;();
    var fresh = new List&lt;string&gt;();
    for (int i = 0; i &lt; TenPullSize; i++)
    {
        var card = roster[_drawCursor % roster.Length];
        _drawCursor++;
        pulled.Add(card.id);
        if (UnlockMet(card) &amp;&amp; !_owned.Contains(card.id))
        {
            _owned.Add(card.id);
            fresh.Add(card.id);
        }
    }
    draw.PulledIds = pulled.ToArray();
    draw.NewIds = fresh.ToArray();
    ApplyPassives();
    _funnel.Add("evt_gacha_landed@" + Stamp());
    return draw;
}
// ---- the passive resolution (the frozen closed set) ----
/// &lt;summary&gt;Re-resolves every owned passive and pushes the factors
/// into the serve engine (zero defaults keep the T005 face
/// unchanged). comedy_flame resolves on the BlazeArmed query face
/// below (the future event spine consumes it); wash_speed_pct and
/// delivery_pct resolve on their own query faces until the dish and
/// takeout spines land.&lt;/summary&gt;
public void ApplyPassives()
{
```

```
    bool hasZencut = false, hasCashier = false, hasTaster = false;
    foreach (var c in RestaurantData.Chefs())
    {
        if (!_owned.Contains(c.id)) continue;
        if (c.passiveId == "serve_speed_pct") hasZencut = true;
        if (c.passiveId == "price_pct") hasCashier = true;
        if (c.passiveId == "research_pct") hasTaster = true;
    }
    _serve.SetCardFactors(AlbumIncomeBonusPct,
        hasCashier ? CashierPricePct : 0,
        hasZencut ? ZencutServePct : 0,
        hasTaster ? TasterResearchPct : 0);
}
/// &lt;summary&gt;The blaze face: the bangs-burn comedy stands armed once
/// the blaze chef is hired (the first event card law, the future
/// spine consumes the face).&lt;/summary&gt;
public bool BlazeArmed =&gt; _owned.Contains("chef_blaze");
/// &lt;summary&gt;The dishwasher face: the slacker's wash factor when the
/// card is owned (0 otherwise; the dish spine consumes it later).&lt;/summary&gt;
public int WashFactorPct =&gt; Owns("chef_dishwasher") ? DishwasherWashPct : 0;
/// &lt;summary&gt;The speedster face: the takeout factor when the card is
/// owned (0 otherwise; the takeout spine consumes it later).&lt;/summary&gt;
public int DeliveryFactorPct =&gt; Owns("chef_speedy") ? SpeedyDeliveryPct : 0;
/// &lt;summary&gt;The taster face (T009a): the R&amp;amp;D factor when the brave
/// taster is owned (0 otherwise; the dark kitchen consumes it on its
/// deterministic success accumulator - 60 becomes exactly 75).&lt;/summary&gt;
public int TasterFactorPct =&gt; Owns("chef_taster") ? TasterResearchPct : 0;
/// &lt;summary&gt;Dark-dish registration (T009a): a researched weird dish
/// joins the dark album through its own spine (never the ten-pull -
/// the dark rows count toward their volume, none are drawable).
/// Idempotent: only a fresh id re-applies the passives so a
/// completed volume books its frozen +2% income bonus the moment it
/// completes.&lt;/summary&gt;
public bool RegisterDarkUnlock(string darkId)
{
    bool known = false;
    foreach (var d in RestaurantData.DarkRecipes()) if (d.id == darkId) known = true;
    if (!known || _owned.Contains(darkId)) return false;
    _owned.Add(darkId);
    ApplyPassives();
    return true;
}
string Stamp() =&gt; _serve.SimTime.ToString("0.00");
/// &lt;summary&gt;Canonical fingerprint: the zero-RNG parity instrument
/// (the same script = the same bytes).&lt;/summary&gt;
public string Fingerprint()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("cursor=").Append(_drawCursor);
    sb.Append("|pulls=").Append(_tenPullsDone);
```

```
            sb.Append("|owned=");
            foreach (var id in _owned) sb.Append(id).Append(',');
            sb.Append("|vols=").Append(CompletedVolumes());
            sb.Append("|bonus=").Append(AlbumIncomeBonusPct);
            sb.Append("|blaze=").Append(BlazeArmed ? 1 : 0);
            sb.Append("|wash=").Append(WashFactorPct);
            sb.Append("|delivery=").Append(DeliveryFactorPct);
            sb.Append("|funnel=").Append(_funnel.Count);
            return sb.ToString();
        }
    }
}
// DarkKitchen.cs
// DarkKitchen.cs - the dark-dish research spine (G14-T009a, SYS07).
// Zero-RNG deterministic research: one shared success accumulator rides
// the frozen 60% law (acc += pct; &gt;= 100 = success, acc -= 100 - the
// long run is exactly 60%, or exactly 75% once the brave taster's +15
// joins the roll). A success lands the weird dish on the dark album
// through the chef-card registration face (idempotent - re-research
// keeps the museum collecting, the never-wasted comedy), becomes the
// signature display, and books +15 on the loved-egg accumulator (the
// "someone genuinely loves it" easter egg fires deterministically at
// &gt;= 100). A failure auto-enters the dark-dish museum with its attempt
// number. The wallet pays on the serve engine's spend side so the
// exact-wallet identity keeps holding. All state in plain fields - the
// check slices construct the engine directly.
using System.Collections.Generic;
namespace Sy.CrazyRestaurant
{
    /// &lt;summary&gt;One museum exhibit: a failed research attempt, auto-entered
    /// (dish + attempt number - the never-wasted collection comedy).&lt;/summary&gt;
    public sealed class MuseumExhibit
    {
        public string dishId = "";
        public int attemptNo;
    }
    /// &lt;summary&gt;One research attempt outcome (the check's assertion face).
    /// Attempted = false means the tap refused (bad index or empty
    /// wallet) and no state moved.&lt;/summary&gt;
    public sealed class DarkResearchResult
    {
        public bool Attempted;
        public bool success;
        public bool freshUnlock;     // true = the dish joined the dark album this attempt
        public bool lovedTriggered; // true = the loved egg fired this attempt
        public string dishId = "";
        public int attemptNo;
    }
    public sealed class DarkKitchen
    {
```

```
readonly RestaurantServe _serve;
readonly ChefCards _cards;
readonly int[] _attempts = new int[RestaurantData.DarkRecipeCount];
readonly List&lt;MuseumExhibit&gt; _museum = new List&lt;MuseumExhibit&gt;();
readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
int _successAcc;          // the shared deterministic success accumulator
int _lovedAcc;            // the loved-egg accumulator (+15 per success)
int _researchCount;       // total paid attempts
int _unlockCount;         // distinct dishes on the dark album
int _lovedCount;          // loved-egg fires so far
string _signatureDishId = "";
public DarkKitchen(RestaurantServe serve, ChefCards cards)
{
    _serve = serve;
    _cards = cards;
}
// ---- live faces ----
public IReadOnlyList&lt;MuseumExhibit&gt; Museum =&gt; _museum;
public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
public int ResearchCount =&gt; _researchCount;
public int UnlockCount =&gt; _unlockCount;
public int LovedCount =&gt; _lovedCount;
public int SuccessAcc =&gt; _successAcc;
public int LovedAcc =&gt; _lovedAcc;
public string SignatureDishId =&gt; _signatureDishId;
public bool LovedTriggered =&gt; _lovedCount &gt; 0;
public int AttemptNoOf(int darkIndex)
    =&gt; darkIndex &gt;= 0 &amp;&amp; darkIndex &lt; _attempts.Length ? _attempts[darkIndex] : 0;
/// &lt;summary&gt;The deterministic success percent of the next attempt:
/// the frozen 60% face plus the taster's R&amp;D factor (exactly 75%
/// with the brave taster hired).&lt;/summary&gt;
public int NextSuccessPct
    =&gt; RestaurantData.DarkResearchSuccessPct + (_cards != null ? _cards.TasterFactorPct : 0);
/// &lt;summary&gt;One research attempt: pays the frozen cost on the
/// wallet spend side, books the attempt, then resolves the shared
/// accumulator (zero RNG - the long run is exact). A success
/// registers the dish on the dark album (idempotent face), becomes
/// the signature display and books the loved accumulator; a
/// failure auto-enters the museum (dish + attempt number, never
/// wasted). Every attempt books its funnel line in order.&lt;/summary&gt;
public DarkResearchResult ResearchAt(int darkIndex)
{
    var res = new DarkResearchResult();
    var table = RestaurantData.DarkRecipes();
    if (darkIndex &lt; 0 || darkIndex &gt;= table.Length) return res;
    if (_serve == null || !_serve.TrySpendDarkResearch(table[darkIndex].researchCost)) return res;
    _researchCount++;
    _attempts[darkIndex]++;
    res.Attempted = true;
    res.dishId = table[darkIndex].id;
```

```
    res.attemptNo = _attempts[darkIndex];
    _funnel.Add("evt_dark_research@" + Stamp());
    _successAcc += NextSuccessPct;
    res.success = _successAcc &gt;= 100;
    if (res.success) _successAcc -= 100;
    if (res.success)
    {
        // the album registration is idempotent: a re-research
        // success keeps the loved face alive (the museum and the
        // easter egg stay reachable) without double-booking.
        res.freshUnlock = _cards != null &amp;&amp; _cards.RegisterDarkUnlock(table[darkIndex].id);
        if (res.freshUnlock)
        {
            _unlockCount++;
            _funnel.Add("evt_dark_unlock@" + Stamp());
        }
        _signatureDishId = table[darkIndex].id;
        _lovedAcc += RestaurantData.LovedEggChancePct;
        if (_lovedAcc &gt;= 100)
        {
            _lovedAcc -= 100;
            _lovedCount++;
            res.lovedTriggered = true;
            _funnel.Add("evt_dark_loved@" + Stamp());
        }
    }
    else if (RestaurantData.MuseumAutoEntry)
    {
        _museum.Add(new MuseumExhibit { dishId = table[darkIndex].id, attemptNo = res.attemptNo });
        _funnel.Add("evt_dark_museum@" + Stamp());
    }
    return res;
}
string Stamp() =&gt; _serve.SimTime.ToString("0.00");
/// &lt;summary&gt;Canonical fingerprint: the zero-RNG parity instrument
/// (the same script = the same bytes).&lt;/summary&gt;
public string Fingerprint()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("research=").Append(_researchCount);
    sb.Append("|unlocks=").Append(_unlockCount);
    sb.Append("|acc=").Append(_successAcc);
    sb.Append("|lovedacc=").Append(_lovedAcc).Append(',').Append(_lovedCount);
    sb.Append("|signature=").Append(_signatureDishId);
    sb.Append("|museum=");
    foreach (var m in _museum) sb.Append(m.dishId).Append('#').Append(m.attemptNo).Append(',');
    sb.Append("|attempts=");
    for (int i = 0; i &lt; _attempts.Length; i++) sb.Append(_attempts[i]).Append(',');
    sb.Append("|funnel=").Append(_funnel.Count);
    return sb.ToString();
```

```
        }
    }
}
// RestaurantAlbum.cs
// RestaurantAlbum.cs - the album aggregator, the honor wall, the signature
// recipe card and the review-literature wall (G14-T011a, SYS08/M10).
// A pure read-only aggregation over the frozen card faces: the three
// album tabs (chefs / customers / dark dishes), the honor wall (one
// golden-wok badge per completed volume), the hand-drawn menu-style
// signature recipe card - the category-unique flex card: the header, the
// latest signature dish of the house, the three volume count lines, the
// word of mouth and the kitchen crew's sign-off - and the second-run
// inheritance face (the 20% gold floor, the full album snapshot, the
// chef team kept, the rank restarting at the street stall). Zero new
// game state and zero changes on ChefCards/RestaurantServe: every face
// reads the live engines (the album law gates itself through collection).
// The share tap books its three-line funnel in order (tap -&gt; card
// composed -&gt; landed - the local zero-server share face, red line 12).
// The review-literature wall stands armed once three reviews sit in the
// collection (the E02 essay spine books evt_review_collected). The
// empire ending stands armed at rank &gt;= R5 (the panorama-light show is
// the view slice's consumption). Pure logic, no MonoBehaviour, zero RNG:
// the same state always fingerprints byte-identical.
using System;
using System.Collections.Generic;
namespace Sy.CrazyRestaurant
{
    /// &lt;summary&gt;One album tab face: the volume id, the owned ids (the
    /// collection order mirrors ChefCards.Owned), the frozen volume size
    /// and the completion flag - a pure query, zero new state.&lt;/summary&gt;
    public sealed class AlbumVolumeFace
    {
        public string id = "";
        public string[] ownedIds = Array.Empty&lt;string&gt;();
        public int size;
        public bool complete;
    }
    /// &lt;summary&gt;The honor wall face: the completed-volume count and the
    /// golden-wok badges - one per completed volume.&lt;/summary&gt;
    public sealed class HonorWallFace
    {
        public int completedVolumes;
        public int goldenWokBadges;
    }
    /// &lt;summary&gt;The second-run carry (M10): the 20% gold floor, the full
    /// album snapshot, the chef team kept and the rank restart. A pure
    /// function face - the new run consumes it, nothing ever mutates for
    /// it.&lt;/summary&gt;
    public sealed class NewRunInheritanceFace
    {
```

```
    public int inheritGoldHc;
    public string[] albumsKept = Array.Empty&lt;string&gt;();
    public bool chefTeamKept;
    public bool rankReset;
}
public sealed class RestaurantAlbum
{
    readonly RestaurantStage _stage;
    readonly ChefCards _cards;
    public RestaurantAlbum(RestaurantStage stage, ChefCards cards = null)
    {
        _stage = stage;
        _cards = cards ?? stage.Cards;
    }
    // ---- face 1: the three album tabs (pure query, owned read-only) ----
    /// &lt;summary&gt;The three frozen tab ids (chefs / customers / dark
    /// dishes).&lt;/summary&gt;
    public static string[] TabIds() =&gt; ChefCards.AlbumVolumeIds();
    /// &lt;summary&gt;One tab face over the frozen volume: the owned ids
    /// mirror ChefCards.OwnedInVolume exactly (the card system stays
    /// the single collection authority - zero new state here, the
    /// T011b gate reconciles the two faces against each other).&lt;/summary&gt;
    public AlbumVolumeFace VolumeFace(string albumId)
    {
        var owned = new List&lt;string&gt;();
        foreach (var id in _cards.Owned)
            if (AlbumOf(id) == albumId) owned.Add(id);
        var face = new AlbumVolumeFace
        {
            id = albumId,
            ownedIds = owned.ToArray(),
            size = _cards.VolumeSize(albumId),
        };
        face.complete = face.ownedIds.Length &gt;= face.size;
        return face;
    }
    /// &lt;summary&gt;The card album mapping, mirrored through the same
    /// frozen tables ChefCards.OwnedInVolume rides (the chef roster
    /// first, then the dark-dish table - the exact two-table law, no
    /// third source so the mirror can never diverge).&lt;/summary&gt;
    static string AlbumOf(string cardId)
    {
        foreach (var c in ChefCards.Roster()) if (c.id == cardId) return c.albumId;
        foreach (var d in RestaurantData.DarkRecipes()) if (d.id == cardId) return d.albumId;
        return "";
    }
    // ---- face 2: the honor wall ----
    /// &lt;summary&gt;The honor wall: the completed volumes and one
    /// golden-wok badge per completed volume.&lt;/summary&gt;
    public HonorWallFace HonorWall()
```

```
{
    int completed = _cards.CompletedVolumes();
    return new HonorWallFace
    {
        completedVolumes = completed,
        goldenWokBadges = completed,
    };
}
// ---- face 3: the signature recipe card (the flex card) ----
/// &lt;summary&gt;The signature dish of the house: the latest researched
/// recipe (the menu order is the research order). Empty when the
/// stall has only its opening dish.&lt;/summary&gt;
public string SignatureDishId
{
    get
    {
        var menu = _stage.Serve.Menu;
        return menu.Count &gt; 1 ? menu[menu.Count - 1] : "";
    }
}
/// &lt;summary&gt;The hand-drawn menu-style card payload: the header, the
/// signature dish of the house, the three volume counts, the word
/// of mouth and the sign-off - composed from the freshest state,
/// byte-identical for the same state (the RestaurantOffline
/// ComposeSummary law). Text keys ride RestaurantTexts with
/// En/zhCN parity.&lt;/summary&gt;
public string ComposeRecipeCard()
{
    var chef = VolumeFace("album_chef");
    var customer = VolumeFace("album_customer");
    var dark = VolumeFace("album_dark");
    var sb = new System.Text.StringBuilder();
    sb.Append(RestaurantTexts.EnText(RestaurantTexts.KeyReportHeader));
    sb.Append('\n').Append(string.Format(RestaurantTexts.EnText(RestaurantTexts.KeyReportSignature),
        SignatureDishId == "" ? "-" : DishNameOf(SignatureDishId)));
    sb.Append('\n').Append(string.Format(RestaurantTexts.EnText(RestaurantTexts.KeyReportAlbums),
        chef.ownedIds.Length, chef.size,
        customer.ownedIds.Length, customer.size,
        dark.ownedIds.Length, dark.size));
    sb.Append('\n').Append(string.Format(RestaurantTexts.EnText(RestaurantTexts.KeyReportWord),
        _stage.Serve.Word));
    sb.Append('\n').Append(RestaurantTexts.EnText(RestaurantTexts.KeyReportFooter));
    return sb.ToString();
}
/// &lt;summary&gt;The share tap: books the three-line funnel in order
/// (tap -&gt; card composed -&gt; landed) and returns the composed card.
/// The share itself stays local (red line 12 - zero server
/// faces).&lt;/summary&gt;
public string TapShare()
{
```

```
    Funnel.Add("evt_album_share_tap@" + Stamp());
    var card = ComposeRecipeCard();
    Funnel.Add("evt_album_share_card@" + Stamp());
    Funnel.Add("evt_album_share_landed@" + Stamp());
    return card;
}
/// &lt;summary&gt;evt_album_share funnel (T011a): the three share lines
/// land in order for the T011b gate assertions.&lt;/summary&gt;
public readonly List&lt;string&gt; Funnel = new List&lt;string&gt;();
string Stamp() =&gt; _stage.Serve.SimTime.ToString("0.00");
// ---- face 4: the review-literature wall (the unlock law) ----
/// &lt;summary&gt;True once the collection holds the frozen three
/// reviews (the E02 essay spine books evt_review_collected - the
/// unlock count rides RestaurantData).&lt;/summary&gt;
public bool ReviewWallArmed
    =&gt; _stage.Events != null &amp;&amp;
       _stage.Events.ReviewsCollected &gt;= RestaurantData.ReviewCollectionUnlockCount;
/// &lt;summary&gt;The review-literature grand-prix long image: armed only
/// once three reviews sit collected, empty before the unlock (the
/// locked window never composes - the view slice renders the
/// armed face). Byte-deterministic from the live state. T028: the
/// armed face carries the genre-signature body - one genre-tagged
/// essay line per collected review (the eight-register family,
/// round-robin rotation, every consecutive pair differs).&lt;/summary&gt;
public string ComposeReviewWall()
{
    if (!ReviewWallArmed) return "";
    string title = string.Format(RestaurantTexts.EnText(RestaurantTexts.KeyReportWallTitle),
        _stage.Events.ReviewsCollected);
    string body = RestaurantReviewGenres.WallBody(_stage.Events.ReviewsCollected);
    return body.Length == 0 ? title : title + "\n" + body;
}
// ---- face 5: the second-run inheritance (M10, pure function) ----
/// &lt;summary&gt;The next-boss carry: the frozen 20% gold floor, the
/// full album snapshot (owned ids keep their order), the chef team
/// kept and the rank restarting at the street stall. Reads the
/// frozen constants - a pure function of its inputs.&lt;/summary&gt;
public static NewRunInheritanceFace NewRunInheritance(int goldHc, ChefCards cards)
{
    var owned = new List&lt;string&gt;();
    if (cards != null)
        foreach (var id in cards.Owned) owned.Add(id);
    return new NewRunInheritanceFace
    {
        inheritGoldHc = goldHc * RestaurantData.InheritGoldPercent / 100,
        albumsKept = owned.ToArray(),
        chefTeamKept = RestaurantData.InheritChefTeamKept,
        rankReset = RestaurantData.InheritRankReset,
    };
}
```

```
        // ---- face 6: the empire ending (armed face) ----
        /// &lt;summary&gt;True once the ladder tops out at R5: the panorama
        /// light show stands armed (the view slice consumes the face -
        /// the all-lit ending beat).&lt;/summary&gt;
        public bool EmpireEndingArmed =&gt; _stage.Serve.Rank.rank &gt;= RestaurantData.RankCount;
        // ---- the copy mirrors (compiled copy surface, zero restated ids) ----
        static readonly string[] RecipeTextKeys =
        {
            RestaurantTexts.KeyRecPlainNoodle, RestaurantTexts.KeyRecBigplateNoodle,
            RestaurantTexts.KeyRecMapoTofuRice,
        };
        static string DishNameOf(string recipeId)
        {
            var recipes = RestaurantData.Recipes();
            for (int i = 0; i &lt; recipes.Length &amp;&amp; i &lt; RecipeTextKeys.Length; i++)
                if (recipes[i].id == recipeId) return RestaurantTexts.EnText(RecipeTextKeys[i]);
            return recipeId;
        }
        /// &lt;summary&gt;Canonical fingerprint: the zero-RNG parity instrument
        /// (the same state = the same bytes - the T011b freeze face
        /// consumes it).&lt;/summary&gt;
        public string Fingerprint()
        {
            var sb = new System.Text.StringBuilder();
            sb.Append("vols=").Append(_cards.CompletedVolumes());
            sb.Append("|sig=").Append(SignatureDishId);
            sb.Append("|report=").Append(ComposeRecipeCard().Length);
            sb.Append("|wall=").Append(ReviewWallArmed ? 1 : 0);
            sb.Append("|ending=").Append(EmpireEndingArmed ? 1 : 0);
            sb.Append("|funnel=").Append(Funnel.Count);
            return sb.ToString();
        }
    }
}
// RestaurantAlbumCard.cs
// RestaurantAlbumCard.cs - the signature recipe share-card view (G14-T011b,
// SYS08/M10). The frozen T011a payload becomes the programmatic card: the
// hand-drawn menu bytes project straight onto the shared ShareCardService
// spec - the header line the title, the signature/albums/word-of-mouth
// lines the body, the kitchen crew's sign-off the footer - and the shared
// service renders the RT card (the X033 four laws live there: the layer
// walk last, Canvas.ForceUpdateCanvases, the manual camera Render, the
// PNG readback) into persistentDataPath. Art-first (X027): the
// g14_sharecard stem ships in S2.5; today the procedural dark card
// carries the flex (a missing stem never breaks a view). Zero new copy
// and zero restated text: every line rides the frozen payload
// (byte-projection - the T011b gate reconciles the two faces against
// each other). The share itself stays local (red line 12 - zero server
// faces: a stable local query, never a round-trip). Pure view, no
// MonoBehaviour.
```

```
using System.Collections.Generic;
using Sy.Social;
namespace Sy.CrazyRestaurant
{
    public static class RestaurantAlbumCard
    {
        /// &lt;summary&gt;Art stem for the share-card template (the S2.5
        /// shipping seat; absent today = the procedural dark card face
        /// of the shared service).&lt;/summary&gt;
        public const string TemplateStem = "g14_sharecard";
        /// &lt;summary&gt;The local share query (red line 12: a stable local
        /// payload riding the platform share face, never a server).&lt;/summary&gt;
        public const string ShareQuery = "g14_recipe";
        /// &lt;summary&gt;The card spec projected from the frozen payload: the
        /// header line the title, the middle lines the body, the sign-off
        /// the footer - the card is a pure projection of
        /// ComposeRecipeCard (zero restated copy, byte-projection).&lt;/summary&gt;
        public static ShareCard SpecFrom(RestaurantAlbum album)
        {
            var spec = new ShareCard
            {
                TemplateKey = TemplateStem,
                Query = ShareQuery,
            };
            string[] lines = album.ComposeRecipeCard().Split('\n');
            if (lines.Length &gt; 0) spec.Title = lines[0];
            var body = new List&lt;string&gt;();
            for (int i = 1; i &lt; lines.Length - 1; i++) body.Add(lines[i]);
            spec.Lines = body.ToArray();
            if (lines.Length &gt; 1) spec.Footer = lines[lines.Length - 1];
            return spec;
        }
        /// &lt;summary&gt;Composes the PNG share card: the RT render and the PNG
        /// readback ride the shared ShareCardService (the tier ladder
        /// keeps the file inside the byte budget); the result carries
        /// Ok/Path/PngBytes/Width/Height - the T011b pixel canary consumes
        /// the written file. The share tap books its own three-line funnel
        /// on the album (TapShare); this face only paints.&lt;/summary&gt;
        public static ComposeResult Compose(RestaurantAlbum album)
        {
            var service = new ShareCardService();
            return service.Compose(SpecFrom(album));
        }
    }
}
// RestaurantBoost.cs
// RestaurantBoost.cs - the research-window boost gate (G14-T015b, SYS10
// boost). The lightbulb moment: while an R&amp;D window runs, the natural
// recommendation slot opens - a tap prechecks the g14_boost 3/day
// category budget, the rewarded view consumes on the same policy
```

```
// instance and the same day bucket, and the completed view lands the
// current research at x2 drain (the frozen x2 family, numeric pack
// section 8) - the window is the research: the completion tick clears
// the factor back to neutral, so the next R&amp;amp;D beat always starts
// fresh. The three-piece law (G08 T009 / FarmBoost T015b precedent):
// the HasCategoryBudget precheck and the ShowRewarded consumption ride
// the same policy instance and the same day bucket (they can never
// disagree), the evt_ad funnel books offer -&gt; show -&gt; complete -&gt;
// boost_landed in order, and a capped tap refuses BEFORE the ad ever
// plays. In-window repeats (boost already landed) refuse pre-offer
// with zero state change (the G11 wage-boost precedent). Pure logic,
// no MonoBehaviour, zero RNG.
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
namespace Sy.CrazyRestaurant
{
    public sealed class RestaurantBoost
    {
        /// &lt;summary&gt;Boost point id (T015b): draws from its own 3/day
        /// g14_boost safety-valve category (AdCategories.CategoryOfPoint
        /// authority, the helper-point precedent - premium-capped so it
        /// never farms).&lt;/summary&gt;
        public const string BoostPoint = "g14_boost";
        /// &lt;summary&gt;The frozen x2 family anchor (numeric pack section 8):
        /// the lightbulb moment doubles the current research window's
        /// drain - the same x2 the festival boost family rides (G13
        /// season-window anchor).&lt;/summary&gt;
        public const int BoostFactorBp = 20000;
        readonly RestaurantStage _stage;
        readonly AdCapPolicy _adCap;
        readonly IAdsService _ads;
        readonly Func&lt;string&gt; _dayKey;
        public RestaurantBoost(RestaurantStage stage, AdCapPolicy adCap, IAdsService ads, Func&lt;string&gt; dayKey = nu
        {
            _stage = stage;
            _adCap = adCap;
            _ads = ads;
            // the precheck reads the very bucket the registered service consumes
            // (injected key only for deterministic gates; production rides TodayKey)
            _dayKey = dayKey ?? AdCapPolicy.TodayKey;
            if (_ads != null) _ads.OnAdEvent += OnAdLifecycle;
        }
        /// &lt;summary&gt;Window face: the research window owns the offer (an
        /// R&amp;amp;D beat in flight = the natural recommendation slot; the
        /// window is the current research).&lt;/summary&gt;
        public bool WindowActive =&gt; _stage != null &amp;&amp; _stage.Serve.ResearchRunning;
        /// &lt;summary&gt;Live boost-category precheck: always-fresh budget face
        /// on the same policy instance and the same day bucket the
```

```
        /// registered service consumes from, so precheck and consume can
        /// never disagree.&lt;/summary&gt;
        public bool OfferBudgetOk
            =&gt; _adCap != null
               &amp;&amp; _adCap.HasCategoryBudget(_dayKey(), AdCategories.CategoryOfPoint(BoostPoint));
        /// &lt;summary&gt;Already landed in this window: repeat taps refuse
        /// pre-offer with zero state change (the G11 wage-boost precedent).&lt;/summary&gt;
        public bool BoostActive =&gt; _stage != null &amp;&amp; _stage.Serve.BoostFactorBp != 10000;
        /// &lt;summary&gt;The boost tap: window guard - already-landed guard -
        /// budget precheck - rewarded view - the x2 lands on the current
        /// research. A dead tap (window closed, boost active, or capped)
        /// burns nothing: no offer entry, no ad, no budget.&lt;/summary&gt;
        public async Task&lt;bool&gt; ClaimBoost()
        {
            if (!WindowActive) return false;
            if (BoostActive) return false;
            if (!OfferBudgetOk)
            {
                Funnel.Add("evt_ad_offer_capped@" + Stamp());   // the honest face refuses at the tap, never after the a
                return false;
            }
            Funnel.Add("evt_ad_offer@" + Stamp());
            RewardResult r = await _ads.ShowRewarded(BoostPoint);
            if (r == null || !r.Completed) return false;   // evt_ad_fail rides the lifecycle face; the retry window sta
            _stage.LandBoost(BoostFactorBp);   // the x2 lands on the serve engine + the stage bus on the same tick
            Funnel.Add("evt_boost_landed@" + Stamp());
            return true;
        }
        /// &lt;summary&gt;evt_ad funnel (T015b): lifecycle events of the boost
        /// point land in the funnel list for gate assertions.&lt;/summary&gt;
        void OnAdLifecycle(AdLifecycle e)
        {
            if (e.PointId != BoostPoint) return;
            Funnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant() + "@" + Stamp());
        }
        public readonly List&lt;string&gt; Funnel = new List&lt;string&gt;();
        string Stamp() =&gt; _stage == null ? "0.00" : _stage.SimTime.ToString("0.00");
    }
}
// RestaurantData.cs
// RestaurantData.cs - frozen data schemas and tables for Crazy Restaurant
// (G14-T004). T007a added the diner event table (E01-E10) - a structural
// move, so the schema version bumped 1 -&gt; 2 with it. The numeric pack doc
// G14 v1 is the source of
// truth and this file is its compiled mirror; Texts/Placeholder/Check
// land in the T004b slice). Schema freeze means the structural laws live
// here as code and DataSchemaCheck proves every law on every row. Values
// are sweep-tunable later, the structure only moves with a schema version
// bump. Zero server faces: the tables ship in the build (red line 12).
// Compliance faces ride the freeze: every dark dish stays a fictional
```

```
// original name, reviews read as two-way comedy collectibles, and the
// delivery rider stays a positive face. Recorded deviation from the G09
// line: offline takeout runs at full pace (doc M09 "orders all taken").
namespace Sy.CrazyRestaurant
{
    /// &lt;summary&gt;One recipe row: the serving economy face (M01) frozen per
    /// fictional dish. Price must clear cost x4 (the healthy margin law -
    /// an idle diner never runs a loss).&lt;/summary&gt;
    public sealed class RecipeDef
    {
        public string id = "";
        public int cookSeconds;    // wok to counter, the three-beat serve
        public int prepCost;       // gold per batch
        public int dishPriceHc;    // sale price in gold hundredths
        public string albumId = "";
    }
    /// &lt;summary&gt;One dark-dish research row (M07): weird-fiction dishes with
    /// monotonic research costs. A success lands on the menu (someone
    /// loves it - the easter egg), a failure auto-enters the museum.&lt;/summary&gt;
    public sealed class DarkRecipeDef
    {
        public string id = "";
        public int researchCost;   // gold per research attempt
        public string albumId = "";
    }
    /// &lt;summary&gt;One chef row (M03): quirky hires, each with one passive from
    /// the closed set - the comedy rides the texts table.&lt;/summary&gt;
    public sealed class ChefDef
    {
        public string id = "";
        public string passiveId = "";   // comedy_flame / serve_speed_pct / price_pct / wash_speed_pct / delivery_pct /
        public string albumId = "";
    }
    /// &lt;summary&gt;One customer row (M05/M08): the review-literature family -
    /// an event source and a collectible at the same time. minRank mirrors
    /// the event wheel's earliest featuring rank (zero drift).&lt;/summary&gt;
    public sealed class CustomerDef
    {
        public string id = "";
        public int minRank;       // the rank the customer first walks in (mirrors the event table)
        public string albumId = "";
    }
    /// &lt;summary&gt;One diner event card row (T007, the comedy spine): the
    /// diner cast drops scene cards - customer rows mirror the customer
    /// table's minRank (zero drift), review rows swing word with the
    /// vlogger anchoring +/-3, comedy cards cap at the +/-2 family band,
    /// the festival row rides the seasonal share face, and chain rows
    /// link into their follow-up card (review_intel / dark_followup).&lt;/summary&gt;
    public sealed class EventDef
    {
```

```
    public string id = "";
    public int rank;            // 0 = any rank (resident), 1-5 = from-rank floor
    public string type = "";    // comedy / review / dark / festival / grand (closed set)
    public int wordSwing;      // 0 or +/-1..3 (the card's word-of-mouth face)
    public string customerId = "";// non-empty when a customer family member stars
    public string chain = "";   // "" / review_intel / dark_followup (closed set)
}
/// &lt;summary&gt;One restaurant rank row: the stove economy (M02) and the
/// income ceiling frozen on one row per rank R1-R5.&lt;/summary&gt;
public sealed class RankRow
{
    public int rank;            // 1-5 (R1 street stall .. R5 global food empire)
    public int stoves;          // preset stove slots
    public int firstStoveCost;  // gold cost of the first stove
    public int stoveOutputHc;   // baseline stove income in gold hundredths per second
    public int incomeCapHc;     // full-rank income ceiling in gold hundredths per second
}
/// &lt;summary&gt;One quality-audit gate (M04, dual track): the hard metrics
/// AND the audit event together - a promotion is a new gameplay
/// ticket, never a wall. Failing an audit only arms the rectification
/// comedy (the auditor cries from one dark dish).&lt;/summary&gt;
public sealed class AuditGateDef
{
    public int fromRank;
    public int toRank;
    public int minStoves;
    public int minDishes;
    public int minWord;
    public string auditEventId = "";
}
/// &lt;summary&gt;One word-of-mouth band row (M06, the second numeric line,
/// 0-100 segments with a visible two-way comedy face per band).&lt;/summary&gt;
public sealed class WordBand
{
    public int low;         // inclusive
    public int high;        // inclusive
    public string id = "";
}
public static class RestaurantData
{
    public const int SchemaVersion = 2;
    public const int RankCount = 5;
    public const int GateCount = 4;       // R1-&gt;R2 .. R4-&gt;R5
    public const int BandCount = 4;       // the word 0-100 partition
    public const int RecipeCount = 3;
    public const int DarkRecipeCount = 4;
    public const int ChefCount = 6;
    public const int CustomerCount = 4;
    public const int EventCount = 10;     // the diner event wheel (T007a)
    public const int WordMin = 0;
```

```
public const int WordMax = 100;
public const int AlbumVolumeCount = 3;   // chefs / customers / dark dishes
// ---- section 1: the serving economy. The first bowl of noodles
// lands in 1s under the &lt;=1.5s first-serve anchor (red line 11
// pacing source); the first research lands &lt;= 8s. ----
public const int FirstServeMaxMs = 1500;        // first coin from the first dish &lt;= 1.5s
public const int FirstResearchSeconds = 8;      // first recipe research &lt;= 8s
public const int FirstEventCardMaxSeconds = 10;// first event card (blaze burns his bangs) &lt;= 10s
public const int P1StallSeconds = 60;          // P1: stall up + first bowl out
public const int P50FirstChefSeconds = 300;     // P50: first quirky chef + first review literature
public const int AdsPerDayMin = 3;            // north-star ad expectation
// ---- section 4: the customer traffic law - the queue is a
// word-of-mouth function. factorBp = 4000 + word * 120, so word 50
// = the par band at exactly 1.0; high word lines the customers up,
// low word floods the review wall (a two-way visible face). ----
public const int CustomerFactorBaseBp = 4000;
public const int CustomerFactorWordBp = 120;
public static int CustomersAt(int baseCustomers, int word)
{
    int w = ClampWord(word);
    return baseCustomers * (CustomerFactorBaseBp + w * CustomerFactorWordBp) / 10000;
}
static int ClampWord(int v)
    =&gt; v &lt; WordMin ? WordMin : (v &gt; WordMax ? WordMax : v);
// ---- section 2: the stove curve laws ----
public const bool PresetSlotsOnly = true;     // pick WHAT opens, never WHERE (no coordinates in the schema)
public const int SlotCostLinearLaw = 1;      // n-th stove cost = firstStoveCost * n (linear, never exponential)
public const int UpgradeOutputPercentPerLevel = 150;  // private-room output *1.5 per level
public const int UpgradeCostPercentPerLevel = 200;    // upgrade cost *2 per level
public const int UpgradeMaxLevel = 3;                // the upgrade ladder caps at 3
public const int PrivateRoomUnlockRank = 4;          // R4+ high-ticket-per-head facility
public const int PrivateRoomTicketFactorBp = 15000;  // private-room per-head ticket x1.5
public static int StoveCost(RankRow row, int stoveIndex)
    =&gt; row.firstStoveCost * stoveIndex * SlotCostLinearLaw;   // 1-based stove index
// ---- section 3: the audit-gate comedy law (never a wall) ----
public const bool RectifyComedyOnFail = true;   // the auditor cries from one dark dish, resubmit next window
// ---- section 4: the word modifier closed set (v1) ----
public const int VloggerWordSwing = 3;       // the vlogger review swings +/-3
public const int EventWordSwingMin = 1;     // comedy events swing +/-1..2
public const int EventWordSwingMax = 2;
public const bool WordAutoDecay = false;     // v1: no automatic decay
// ---- section 5: the dark-kitchen laws ----
public const int DarkResearchSuccessPct = 60;    // a weird dish lands on the menu six times in ten
public const int LovedEggChancePct = 15;        // someone genuinely loves it (the easter egg)
public const bool MuseumAutoEntry = true;       // a failed dish auto-enters the dark-dish museum (never wasted)
public const int ReviewCollectionUnlockCount = 3;  // three collected reviews unlock the review-literature long
public const int AlbumBonusPerVolumePct = 2;        // each completed volume = income +2% (v1 proposal)
// ---- section 9: the G11 cameo reservation (S3+ LiveOps window) ----
public const string G11CameoReservedKey = "g11_office_cameo";
// ---- section 7: offline (SYS09 - takeout keeps taking orders) ----
```

```
public const int OfflineServePercent = 100;       // offline takeout runs at full pace (doc M09, recorded deviat
public const int OfflineCapHours = 12;            // per-absence cap
public const string OfflineDoubleCategory = "double"; // AdCap family category
public const int DoubleOfferPerSpan = 1;          // one double offer per absence span
public const int DoubleWinMinPer200 = 1;          // frequency window 1-5 per 200 (family law)
public const int DoubleWinMaxPer200 = 5;
// ---- section 8: the AdCap category closed set ----
public const string GachaCategory = "gacha";      // ten-pull chef/recipe draws
public const int GachaWinMinPer200 = 1;
public const int GachaWinMaxPer200 = 5;
public const string BoostCategory = "boost";      // research-window boosts
public const int BoostWinMinPer200 = 1;
public const int BoostWinMaxPer200 = 5;
// ---- section 7: second-run inheritance (M10, the empire ending) ----
public const int InheritGoldPercent = 20;        // floor
public const bool InheritChefTeamKept = true;   // the chef crew follows the new boss
public const bool InheritAlbumsKept = true;     // the three albums persist (the flex face)
public const bool InheritRankReset = true;      // the spine restarts at the street stall
/// &lt;summary&gt;Rank names (English telemetry face; the zh copy rides the texts table).&lt;/summary&gt;
public static readonly string[] RankNames =
    { "STREET STALL", "HOLE-IN-THE-WALL", "OLD TOWN DINER", "CHAIN BANQUET HALL", "GLOBAL FOOD EMPIRE" };
// ---- the recipe table: serving economy (fast first bowl, healthy margins) ----
static readonly RecipeDef[] RecipeTable =
{
    new RecipeDef { id = "rec_plain_noodle",     cookSeconds =  1, prepCost =  5, dishPriceHc =   40, albumId =
    new RecipeDef { id = "rec_bigplate_noodle",  cookSeconds = 30, prepCost = 20, dishPriceHc =  150, albumId =
    new RecipeDef { id = "rec_mapo_tofu_rice",  cookSeconds = 45, prepCost = 40, dishPriceHc =  320, albumId = "
};
public static RecipeDef[] Recipes() =&gt; (RecipeDef[])RecipeTable.Clone();
// ---- the dark-dish research table (M07, weird-fiction originals only) ----
static readonly DarkRecipeDef[] DarkRecipeTable =
{
    new DarkRecipeDef { id = "dark_banlangen_noodle",  researchCost = 100, albumId = "album_dark" },  // banlang
    new DarkRecipeDef { id = "dark_strawberry_pork",   researchCost = 250, albumId = "album_dark" },  // strawbe
    new DarkRecipeDef { id = "dark_childhood_set",      researchCost = 500, albumId = "album_dark" },  // the ch
    new DarkRecipeDef { id = "dark_snail_cup",         researchCost = 900, albumId = "album_dark" },  // snail-n
};
public static DarkRecipeDef[] DarkRecipes() =&gt; (DarkRecipeDef[])DarkRecipeTable.Clone();
// ---- the quirky chef table (M03) ----
static readonly ChefDef[] ChefTable =
{
    new ChefDef { id = "chef_blaze",       passiveId = "comedy_flame",  albumId = "album_chef" },  // the blaze
    new ChefDef { id = "chef_zencut",      passiveId = "serve_speed_pct", albumId = "album_chef" },  // zen prep
    new ChefDef { id = "chef_cashier",     passiveId = "price_pct",     albumId = "album_chef" },  // the cashie
    new ChefDef { id = "chef_dishwasher",  passiveId = "wash_speed_pct", albumId = "album_chef" },  // the slack
    new ChefDef { id = "chef_speedy",      passiveId = "delivery_pct",  albumId = "album_chef" },  // the takeou
    new ChefDef { id = "chef_taster",     passiveId = "research_pct",  albumId = "album_chef" },  // the brave t
};
public static ChefDef[] Chefs() =&gt; (ChefDef[])ChefTable.Clone();
// ---- the review-literature customer table (M05/M08) ----
```

```
static readonly CustomerDef[] CustomerTable =
{
    new CustomerDef { id = "cust_review_writer",     minRank = 1, albumId = "album_customer" },  // the review-l
    new CustomerDef { id = "cust_freebie_actor",      minRank = 2, albumId = "album_customer" },  // the freebie
    new CustomerDef { id = "cust_vlogger",           minRank = 3, albumId = "album_customer" },  // the vlogger
    new CustomerDef { id = "cust_midnight_stranger", minRank = 4, albumId = "album_customer" },  // the midnight
};
public static CustomerDef[] Customers() =&gt; (CustomerDef[])CustomerTable.Clone();
// ---- the diner event table (T007a data layer, E01-E10) ----
static readonly EventDef[] EventTable =
{
    new EventDef { id = "evt_blaze_bangs",        rank = 0, type = "comedy",   wordSwing =  0, customerId = "",
    new EventDef { id = "evt_review_essay",       rank = 1, type = "review",   wordSwing = -1, customerId = "cus
    new EventDef { id = "evt_freebie_actor",      rank = 2, type = "comedy",   wordSwing =  2, customerId = "cus
    new EventDef { id = "evt_dark_first_boom",    rank = 2, type = "dark",     wordSwing =  0, customerId = "",
    new EventDef { id = "evt_vlogger_visit",     rank = 3, type = "review",   wordSwing = -3, customerId = "cust
    new EventDef { id = "evt_dark_second_hit",    rank = 3, type = "dark",     wordSwing =  1, customerId = "",
    new EventDef { id = "evt_foodfest_challenge", rank = 4, type = "festival", wordSwing =  2, customerId = "",
    new EventDef { id = "evt_midnight_stranger",  rank = 4, type = "comedy",   wordSwing =  2, customerId = "cus
    new EventDef { id = "evt_zencut_artwork",     rank = 4, type = "comedy",   wordSwing =  1, customerId = "",
    new EventDef { id = "evt_audit_grand",        rank = 5, type = "grand",    wordSwing =  0, customerId = "cus
};
public static EventDef[] Events() =&gt; (EventDef[])EventTable.Clone();
// ---- the rank table: stove economy + income ceiling on one row ----
static readonly RankRow[] RankTable =
{
    new RankRow { rank = 1, stoves =  1, firstStoveCost =   50, stoveOutputHc =  40, incomeCapHc =  120 },
    new RankRow { rank = 2, stoves =  2, firstStoveCost =  120, stoveOutputHc =  60, incomeCapHc =  300 },
    new RankRow { rank = 3, stoves =  4, firstStoveCost =  300, stoveOutputHc =  90, incomeCapHc =  700 },
    new RankRow { rank = 4, stoves =  6, firstStoveCost =  700, stoveOutputHc = 130, incomeCapHc = 1400 },
    new RankRow { rank = 5, stoves = 10, firstStoveCost = 1500, stoveOutputHc = 180, incomeCapHc = 2500 },
};
public static RankRow[] Ranks() =&gt; (RankRow[])RankTable.Clone();
// ---- quality-audit gates (dual track; the audit names ride the
// texts table, the events arm the rectification comedy) ----
static readonly AuditGateDef[] GateTable =
{
    new AuditGateDef { fromRank = 1, toRank = 2, minStoves = 1, minDishes =  3, minWord = 30, auditEventId = "ev
    new AuditGateDef { fromRank = 2, toRank = 3, minStoves = 2, minDishes =  6, minWord = 45, auditEventId = "ev
    new AuditGateDef { fromRank = 3, toRank = 4, minStoves = 4, minDishes = 10, minWord = 55, auditEventId = "ev
    new AuditGateDef { fromRank = 4, toRank = 5, minStoves = 6, minDishes = 14, minWord = 65, auditEventId = "ev
};
public static AuditGateDef[] Gates() =&gt; (AuditGateDef[])GateTable.Clone();
// ---- word bands (the 0-100 partition, four visible faces) ----
static readonly WordBand[] BandTable =
{
    new WordBand { low = 0,  high = 24,  id = "word_bad" },    // the review-literature flood + sharp vlogger ta
    new WordBand { low = 25, high = 49,  id = "word_plain" },  // slow lunch hours, business as usual
    new WordBand { low = 50, high = 74,  id = "word_good" },   // queues out the door
    new WordBand { low = 75, high = 100, id = "word_hot" },    // the food show visits + buyout offers
```

```
};
public static WordBand[] Bands() =&gt; (WordBand[])BandTable.Clone();
public static WordBand BandFor(int word)
{
    foreach (var b in BandTable)
        if (word &gt;= b.low &amp;&amp; word &lt;= b.high) return b;
    return BandTable[0];
}
/// &lt;summary&gt;Canonical signature - the dual-derive byte-parity
/// instrument (the schema freeze proof): two derives of the frozen
/// tables agree here or nowhere.&lt;/summary&gt;
public static string Signature()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("v").Append(SchemaVersion);
    sb.Append("|recipes:");
    foreach (var r in RecipeTable)
        sb.Append(r.id).Append('/')
          .Append(r.cookSeconds).Append(',').Append(r.prepCost).Append(',').Append(r.dishPriceHc).Append(';');
    sb.Append("|ranks:");
    foreach (var r in RankTable)
        sb.Append(r.rank).Append('/')
          .Append(r.stoves).Append(',').Append(r.firstStoveCost).Append(',')
          .Append(r.stoveOutputHc).Append(',').Append(r.incomeCapHc).Append(';');
    sb.Append("|pacing:").Append(FirstServeMaxMs).Append(',').Append(FirstResearchSeconds).Append(',')
      .Append(FirstEventCardMaxSeconds).Append(',').Append(P1StallSeconds).Append(',').Append(P50FirstChefSecond
    sb.Append("|traffic:").Append(CustomerFactorBaseBp).Append('+').Append(CustomerFactorWordBp);
    sb.Append("|stoves:").Append(PresetSlotsOnly).Append(',').Append(SlotCostLinearLaw).Append(',')
      .Append(UpgradeOutputPercentPerLevel).Append(',').Append(UpgradeCostPercentPerLevel).Append(',').Append(Up
      .Append(PrivateRoomUnlockRank).Append(',').Append(PrivateRoomTicketFactorBp);
    sb.Append("|gate:").Append(RectifyComedyOnFail);
    sb.Append("|word:").Append(VloggerWordSwing).Append(',').Append(EventWordSwingMin).Append(',')
      .Append(EventWordSwingMax).Append(',').Append(WordAutoDecay);
    foreach (var b in BandTable) sb.Append(',').Append(b.id).Append(':').Append(b.low).Append('-').Append(b.high
    sb.Append("|dark:").Append(DarkResearchSuccessPct).Append(',').Append(LovedEggChancePct).Append(',')
      .Append(MuseumAutoEntry).Append(',').Append(ReviewCollectionUnlockCount);
    sb.Append("|album:").Append(AlbumVolumeCount).Append(',').Append(AlbumBonusPerVolumePct);
    sb.Append("|cameo:").Append(G11CameoReservedKey);
    sb.Append("|offline:").Append(OfflineServePercent).Append(',').Append(OfflineCapHours).Append(',')
      .Append(OfflineDoubleCategory).Append(',').Append(DoubleOfferPerSpan).Append(',')
      .Append(DoubleWinMinPer200).Append(',').Append(DoubleWinMaxPer200);
    sb.Append("|adcap:").Append(GachaCategory).Append(',').Append(GachaWinMinPer200).Append(',').Append(GachaWin
      .Append(BoostCategory).Append(',').Append(BoostWinMinPer200).Append(',').Append(BoostWinMaxPer200);
    sb.Append("|inherit:").Append(InheritGoldPercent).Append(',').Append(InheritChefTeamKept).Append(',')
      .Append(InheritAlbumsKept).Append(',').Append(InheritRankReset);
    sb.Append("|gates:");
    foreach (var g in GateTable)
        sb.Append(g.fromRank).Append('&gt;').Append(g.toRank).Append('/')
          .Append(g.minStoves).Append(',').Append(g.minDishes).Append(',').Append(g.minWord)
          .Append(',').Append(g.auditEventId).Append(';');
```

```
            sb.Append("|chefs:");
            foreach (var c in ChefTable) sb.Append(c.id).Append('@').Append(c.passiveId).Append(';');
            sb.Append("|customers:");
            foreach (var c in CustomerTable) sb.Append(c.id).Append('/').Append(c.minRank).Append(';');
            sb.Append("|darkrecipes:");
            foreach (var d in DarkRecipeTable) sb.Append(d.id).Append('/').Append(d.researchCost).Append(';');
            sb.Append("|events:");
            foreach (var e in EventTable)
                sb.Append(e.id).Append('/').Append(e.rank).Append('@').Append(e.type)
                  .Append('~').Append(e.wordSwing).Append('+').Append(e.customerId).Append('#').Append(e.chain).Append('
            return sb.ToString();
        }
    }
}
// RestaurantEventEngine.cs
// RestaurantEventEngine.cs - the diner event wheel engine (G14-T007b, SYS04).
// Series-shared SYS04 family asset re-skinned from the G08 CampusEventEngine
// paradigm (via the G13 FarmEventEngine mirror): table-agnostic pacing
// laws, the pool cursor is a deterministic rotation (zero RNG), zero
// wall-clock, zero server faces (red line 12). One clock rides the
// injected serve sim: the card rotation drops the diner cast's scene
// cards - the blaze-bangs first laugh inside the &lt;=10s red-line window
// (the review-intel preview rides the same beat, the opening three-beat
// face), then one card per the 240s cadence band. Word-of-mouth swings
// book on the serve engine (the vlogger anchors +/-3), E02 books the
// review album's first collectible (evt_review_collected), E04 chains
// the dark follow-up card E06 (the museum's second exhibit beat lands
// same-tick), and E10 reveals the undercover review writer (the empire
// audit's grand face). Chain beats other than the closed families fail
// loud at schedule time - never silently dropped.
using System;
using System.Collections.Generic;
namespace Sy.CrazyRestaurant
{
    public enum RestaurantEventKind { Card, ChainStep }
    /// &lt;summary&gt;One logged event beat: a diner scene card or a chain step
    /// (the review-intel preview rides its card same-tick).&lt;/summary&gt;
    public sealed class RestaurantEventLogEntry
    {
        public double t;
        public string id = "";
        public RestaurantEventKind kind;
        public string step = "";        // chain step tag, "" on the card itself
        public int wordSwing;           // the card's word-of-mouth face (chain steps carry zero)
        public string customerId = "";  // non-empty when a customer family member stars
    }
    public sealed class RestaurantEventEngine
    {
        // pacing baselines (the only-up law: these may tighten, never loosen)
        public const double FirstCardDelaySeconds = RestaurantStage.FirstEventCardAt;  // 9.5s: blaze bangs + review int
```

```
public const double CardCadenceSeconds = 240;  // later cards: one per the 3-5 min band
public const double CadenceBandMin = 180;
public const double CadenceBandMax = 300;
public const double ChainStepDelay = 0.0;      // diner chains land same-tick (the preview rides the 9.5s card b
public const string DarkFollowupCardId = "evt_dark_second_hit";  // E06: the chained follow-up of E04 (frozen ta
readonly RestaurantServe _serve;
readonly EventDef[] _table;
readonly List&lt;RestaurantEventLogEntry&gt; _log = new List&lt;RestaurantEventLogEntry&gt;();
readonly List&lt;ChainBeat&gt; _pending = new List&lt;ChainBeat&gt;();
readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
int _rank;
int _cardCursor;
int _fired;
int _reviewsCollected;   // E02 firings: the review-literature album entries
int _undercoverReveals;  // E10 firings: the empire audit's undercover face
double _simTime;
double _nextCardAt;
struct ChainBeat
{
    public double at;
    public string id;
    public string step;
}
/// &lt;summary&gt;Series wiring face: inject the serve engine (the
/// word-of-mouth holder) and the starting rank.&lt;/summary&gt;
public RestaurantEventEngine(RestaurantServe serve, int startRank)
{
    _serve = serve;
    _table = RestaurantData.Events();
    _rank = ClampRank(startRank);
    _nextCardAt = FirstCardDelaySeconds;
}
static int ClampRank(int r)
    =&gt; r &lt; 1 ? 1 : (r &gt; RestaurantData.RankCount ? RestaurantData.RankCount : r);
// ---- live faces ----
public int Rank =&gt; _rank;
public int Fired =&gt; _fired;
public int ReviewsCollected =&gt; _reviewsCollected;
public int UndercoverReveals =&gt; _undercoverReveals;
public double SimTime =&gt; _simTime;
public double NextCardAt =&gt; _nextCardAt;
public IList&lt;RestaurantEventLogEntry&gt; Log =&gt; _log;
public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
/// &lt;summary&gt;Rank moves re-scope the pool live; the rotation cursor
/// keeps running so a mid-journey boss never sees the pool restart.&lt;/summary&gt;
public void SetRank(int r) =&gt; _rank = ClampRank(r);
/// &lt;summary&gt;The card pool: the rank's scene rows (rank 0 = resident
/// any-rank rows, 1-5 = from-rank floor - the customer table's
/// minRank mirror owns the zero-drift law, DataSchemaCheck proves it).&lt;/summary&gt;
public EventDef[] CardPool()
```

```
{
    var pool = new List&lt;EventDef&gt;();
    for (int i = 0; i &lt; _table.Length; i++)
        if (RankScopes(_table[i])) pool.Add(_table[i]);
    return pool.ToArray();
}
bool RankScopes(EventDef e) =&gt; e.rank == 0 || e.rank &lt;= _rank;
/// &lt;summary&gt;Clock tick: due chain beats land first (same-tick beats
/// drain immediately), then any due card - big dt jumps drain every
/// due beat in schedule order, deterministic.&lt;/summary&gt;
public void Advance(double dt)
{
    if (dt &lt; 0) dt = 0;
    double target = _simTime + dt;
    bool progressed = true;
    while (progressed)
    {
        progressed = false;
        double nextChainAt = double.MaxValue;
        for (int i = 0; i &lt; _pending.Count; i++)
        {
            if (_pending[i].at &lt;= target)
            {
                var beat = _pending[i];
                _pending.RemoveAt(i);
                i--;
                LandChainBeat(beat);
                progressed = true;
            }
            else if (_pending[i].at &lt; nextChainAt) nextChainAt = _pending[i].at;
        }
        if (_nextCardAt &lt;= target &amp;&amp; CardPool().Length &gt; 0)
        {
            FireCard(target);
            progressed = true;
        }
    }
    _simTime = target;
}
void FireCard(double target)
{
    var pool = CardPool();
    var def = pool[_cardCursor % pool.Length];
    _cardCursor++;
    _fired++;
    double at = _nextCardAt &lt;= target ? _nextCardAt : target;
    _log.Add(new RestaurantEventLogEntry
    {
        t = at,
        id = def.id,
```

```
            kind = RestaurantEventKind.Card,
            wordSwing = def.wordSwing,
            customerId = def.customerId
        });
        _funnel.Add("evt_event_card@" + at.ToString("0.00"));
        if (def.wordSwing != 0)
            _serve.SetWord(_serve.Word + def.wordSwing);
        if (def.id == "evt_review_essay")
        {
            // E02: the review essay enters the album - the first collectible
            _reviewsCollected++;
            _funnel.Add("evt_review_collected@" + at.ToString("0.00"));
        }
        if (def.id == "evt_audit_grand")
        {
            // E10: the empire audit reveals the undercover review writer
            _undercoverReveals++;
            _funnel.Add("evt_undercover_reveal@" + at.ToString("0.00"));
        }
        if (def.chain.Length &gt; 0) ScheduleChain(def, at);
        _nextCardAt = at + CardCadenceSeconds;
    }
    /// &lt;summary&gt;Chain law: the descriptor names the family. The
    /// review-intel family books the bad-review preview beat that rides
    /// the blaze-bangs card (the 9.5s three-beat face); the dark-followup
    /// family books the chained E06 card (the museum's second exhibit
    /// beat, a full card with its own word face). Any other descriptor
    /// fails loud at schedule time - never silently dropped.&lt;/summary&gt;
    void ScheduleChain(EventDef def, double cardAt)
    {
        string step;
        if (def.chain == "review_intel") step = "review_preview";
        else if (def.chain == "dark_followup") step = "dark_followup_card";
        else throw new InvalidOperationException("unknown chain family: " + def.chain);
        _pending.Add(new ChainBeat { at = cardAt + ChainStepDelay, id = def.id, step = step });
    }
    /// &lt;summary&gt;Chain beat landing: the review preview is pure telemetry
    /// (the card fx already carries the comedy); the dark follow-up
    /// fires the chained E06 card as a full card - its own log entry,
    /// word swing, and funnel beat (the chain realization face).&lt;/summary&gt;
    void LandChainBeat(ChainBeat beat)
    {
        if (beat.step == "review_preview")
        {
            _log.Add(new RestaurantEventLogEntry { t = beat.at, id = beat.id, kind = RestaurantEventKind.ChainStep,
            _funnel.Add("evt_review_intel@" + beat.at.ToString("0.00"));
            return;
        }
        if (beat.step == "dark_followup_card")
        {
```

```
// modifier closed set), books evt_promote and the HQ stamp ceremony
// card (city-wide fireworks + the serving-bell chorus, the ceremony
// data face); a failing audit settles the -3 swing, arms the
// rectification comedy (RectifyComedyOnFail: the auditor cries one
// bite into a dark dish) and books evt_inspect. The word band rides
// BandFor - the two-way visible face: high word lines the customers
// up, low word floods the review wall. The serve-side economy settle
// (PromoteTo) is the T008b wiring slice - zero calls here keeps the
// T005/T007 frozen faces constructively drift-free. Zero RNG, zero
// wall-clock, zero server (red line 12).
using System;
using System.Collections.Generic;
namespace Sy.CrazyRestaurant
{
    public enum RestaurantRankBeatKind { Promote, Inspect }
    /// &lt;summary&gt;One logged rank beat: the promote stamp (an audit passed)
    /// or the inspect fail (the rectification comedy armed).&lt;/summary&gt;
    public sealed class RestaurantRankBeat
    {
        public double t;
        public int fromRank;
        public int toRank;              // = fromRank on a fail (no ladder move)
        public RestaurantRankBeatKind kind;
        public string auditEventId = ""; // the quality-audit face this beat settled under
        public int wordSwing;           // +3 pass / -3 fail (the frozen review anchor)
        public bool promoted;
        public string failDimension = "";// "stoves" / "dishes" / "word" on a fail, "" on a pass
    }
    /// &lt;summary&gt;The HQ stamp ceremony card: the promotion celebration data
    /// face - the stamped seal, the city-wide fireworks, the serving-bell
    /// chorus.&lt;/summary&gt;
    public sealed class RestaurantAuditCeremonyCard
    {
        public double t;
        public int toRank;
        public string rankName = "";         // English telemetry face (zh rides the texts table)
        public string auditEventId = "";
        public string stampKey = "";         // the ceremony banner copy key
    }
    /// &lt;summary&gt;The rectification comedy card: armed on a failed audit -
    /// the auditor cries one bite into a dark dish so the next window
    /// holds (never a wall, a comedy).&lt;/summary&gt;
    public sealed class RestaurantRectifyCard
    {
        public double t;
        public int fromRank;
        public string auditEventId = "";
        public string failDimension = "";
        public string armedKey = "";         // the armed-beat copy key
    }
```

```
public sealed class RestaurantRankEngine
{
    readonly RestaurantServe _serve;   // stoves + word live here
    readonly ChefCards _cards;          // the dish-album dimension lives here
    readonly List&lt;RestaurantRankBeat&gt; _beats = new List&lt;RestaurantRankBeat&gt;();
    readonly List&lt;RestaurantAuditCeremonyCard&gt; _ceremonies = new List&lt;RestaurantAuditCeremonyCard&gt;();
    readonly List&lt;RestaurantRectifyCard&gt; _rectifies = new List&lt;RestaurantRectifyCard&gt;();
    readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
    int _rank;
    bool _rectifyArmed;
    string _armedDimension = "";
    /// &lt;summary&gt;Series wiring face: inject the serving engine (the
    /// stoves and word holder) and the card system (the dish-album
    /// holder).&lt;/summary&gt;
    public RestaurantRankEngine(RestaurantServe serve, ChefCards cards, int startRank)
    {
        _serve = serve;
        _cards = cards;
        _rank = ClampRank(startRank);
    }
    static int ClampRank(int r)
        =&gt; r &lt; 1 ? 1 : (r &gt; RestaurantData.RankCount ? RestaurantData.RankCount : r);
    // ---- live faces ----
    public int Rank =&gt; _rank;
    public bool RectifyArmed =&gt; _rectifyArmed;
    public string ArmedDimension =&gt; _armedDimension;
    public IList&lt;RestaurantRankBeat&gt; Beats =&gt; _beats;
    public IList&lt;RestaurantAuditCeremonyCard&gt; Ceremonies =&gt; _ceremonies;
    public IList&lt;RestaurantRectifyCard&gt; Rectifies =&gt; _rectifies;
    public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
    /// &lt;summary&gt;The frozen gate row for a from-rank (the GateTable
    /// mirror); null past the ladder top (R5 has no gate out).&lt;/summary&gt;
    public AuditGateDef GateFor(int fromRank)
    {
        foreach (var g in RestaurantData.Gates())
            if (g.fromRank == fromRank) return g;
        return null;
    }
    // ---- the audit-gate solver (hard metrics against the live diner) ----
    /// &lt;summary&gt;Dish-album entries collected across every volume (the
    /// gate's dishes dimension: chefs + customers + dark dishes).&lt;/summary&gt;
    public int AlbumCollected()
    {
        int n = 0;
        foreach (var v in ChefCards.AlbumVolumeIds())
            n += _cards.OwnedInVolume(v);
        return n;
    }
    public bool StovesMet(AuditGateDef gate) =&gt; _serve.OpenStoves &gt;= gate.minStoves;
    public bool DishesMet(AuditGateDef gate) =&gt; AlbumCollected() &gt;= gate.minDishes;
```

```
public bool WordMet(AuditGateDef gate) =&gt; _serve.Word &gt;= gate.minWord;
/// &lt;summary&gt;The failing dimension on a miss - "stoves" / "dishes" /
/// "word" in table order, "" when the gate passes.&lt;/summary&gt;
public string FailingDimension(AuditGateDef gate)
{
    if (!StovesMet(gate)) return "stoves";
    if (!DishesMet(gate)) return "dishes";
    if (!WordMet(gate)) return "word";
    return "";
}
// ---- the audit transaction (the promotion engine heart) ----
/// &lt;summary&gt;Runs one quality audit against the from-rank's frozen
/// gate: pass = the +3 settle, evt_promote, the HQ stamp ceremony
/// card and the ladder move; fail = the -3 settle, the
/// rectification comedy armed and evt_inspect. Word settles
/// through the serving engine so the 0-100 clamp law owns every
/// swing. The swing rides the frozen +/-3 review anchor (the
/// vlogger law - the strongest review in the modifier closed set;
/// an audit is a review). The economy row settle is T008b.&lt;/summary&gt;
public RestaurantRankBeat Review(double t, int fromRank)
{
    var gate = GateFor(fromRank);
    if (gate == null)
        throw new InvalidOperationException("no gate from rank " + fromRank);
    string fail = FailingDimension(gate);
    bool pass = fail.Length == 0;
    var beat = new RestaurantRankBeat
    {
        t = t,
        fromRank = gate.fromRank,
        toRank = pass ? gate.toRank : gate.fromRank,
        kind = pass ? RestaurantRankBeatKind.Promote : RestaurantRankBeatKind.Inspect,
        auditEventId = gate.auditEventId,
        wordSwing = pass ? RestaurantData.VloggerWordSwing : -RestaurantData.VloggerWordSwing,
        promoted = pass,
        failDimension = fail
    };
    _serve.SetWord(_serve.Word + beat.wordSwing);
    if (pass)
    {
        _rank = ClampRank(gate.toRank);
        _serve.PromoteTo(gate.toRank);   // the economy row moves with the ladder (the ticket lands, T008b settl
        _ceremonies.Add(new RestaurantAuditCeremonyCard
        {
            t = t,
            toRank = gate.toRank,
            rankName = RestaurantData.RankNames[gate.toRank - 1],
            auditEventId = gate.auditEventId,
            stampKey = RestaurantTexts.KeyGatePromote
        });
```

```
            _funnel.Add("evt_promote@" + t.ToString("0.00"));
            _rectifyArmed = false;
            _armedDimension = "";
        }
        else
        {
            if (RestaurantData.RectifyComedyOnFail)
            {
                _rectifyArmed = true;
                _armedDimension = fail;
                _rectifies.Add(new RestaurantRectifyCard
                {
                    t = t,
                    fromRank = gate.fromRank,
                    auditEventId = gate.auditEventId,
                    failDimension = fail,
                    armedKey = RestaurantTexts.KeyGateRectify
                });
            }
            _funnel.Add("evt_inspect@" + t.ToString("0.00"));
        }
        _beats.Add(beat);
        return beat;
    }
    // ---- the word band two-way face (SYS06) ----
    /// &lt;summary&gt;The live band id (the frozen 0-100 partition).&lt;/summary&gt;
    public string CurrentBandId() =&gt; RestaurantData.BandFor(_serve.Word).id;
    /// &lt;summary&gt;The band's visible two-way face copy - the band id rides
    /// the texts table (word_bad .. word_hot), a miss degrades to the
    /// key itself, never a crash.&lt;/summary&gt;
    public string CurrentBandFace() =&gt; RestaurantTexts.EnText(CurrentBandId());
    /// &lt;summary&gt;The customer traffic at the live word - the queue face
    /// of the same band (high word lines the customers up, low word
    /// floods the review wall).&lt;/summary&gt;
    public int CustomersForCurrentWord(int baseCustomers)
        =&gt; RestaurantData.CustomersAt(baseCustomers, _serve.Word);
    /// &lt;summary&gt;Clears the armed rectification comedy once the
    /// auditor's tears landed (the stage layer's comedy beat consumes
    /// the arm).&lt;/summary&gt;
    public void ConsumeRectifyArm()
    {
        _rectifyArmed = false;
        _armedDimension = "";
    }
    // ---- query faces (the check layer's deterministic probes) ----
    public int CountBeats(RestaurantRankBeatKind kind)
    {
        int n = 0;
        for (int i = 0; i &lt; _beats.Count; i++)
            if (_beats[i].kind == kind) n++;
```

```
            return n;
        }
        public double TimeOfBeat(RestaurantRankBeatKind kind)
        {
            for (int i = 0; i &lt; _beats.Count; i++)
                if (_beats[i].kind == kind) return _beats[i].t;
            return -1.0;
        }
        /// &lt;summary&gt;Deterministic fingerprint - the double-run byte-parity
        /// face (zero-RNG family): identical drives land on identical bytes.&lt;/summary&gt;
        public string RankSig()
        {
            var sb = new System.Text.StringBuilder();
            sb.Append("rank=").Append(_rank);
            sb.Append("|armed=").Append(_rectifyArmed ? 1 : 0).Append(':').Append(_armedDimension);
            for (int i = 0; i &lt; _beats.Count; i++)
            {
                var b = _beats[i];
                sb.Append(";").Append(b.auditEventId).Append('~')
                  .Append(b.fromRank).Append('&gt;').Append(b.toRank)
                  .Append('@').Append(b.t.ToString("0.###"))
                  .Append(b.promoted ? "#promote" : "#inspect").Append(b.promoted ? "" : ":" + b.failDimension)
                  .Append("w").Append(b.wordSwing);
            }
            sb.Append("|cer=").Append(_ceremonies.Count)
              .Append(",rec=").Append(_rectifies.Count)
              .Append("|funnel=").Append(_funnel.Count);
            return sb.ToString();
        }
    }
}
// RestaurantReviewGenres.cs
// RestaurantReviewGenres.cs - the review-literature genre-signature family
// (G14-T028, the U176 review-literature comedy engine). The collected E02
// essays expand from one static line into an eight-register family: every
// collected review binds a genre-signed essay (the comedy law: a deadpan
// register riding absurd content), and the grand-prix wall composes one
// genre-tagged line per collected essay. The rotation cursor is a plain
// round-robin over the frozen order - consecutive picks never land the same
// genre (max run length 1, inside the &lt;=2 anti-streak ceiling of the
// anti-cheap-law), and the composition is byte-deterministic from the
// collected count (pure function - zero RNG, zero state).
// Tuanjie.exe -batchmode -quit -executeMethod Sy.CrazyRestaurant.ReviewGenreCheck.Run
// (assertions live in the Editor check; this file stays pure data + math).
using System.Text;
namespace Sy.CrazyRestaurant
{
    public static class RestaurantReviewGenres
    {
        /// &lt;summary&gt;The frozen genre count (the taskbook floor: &gt;=8).&lt;/summary&gt;
```

```
public const int GenreCount = 8;
/// &lt;summary&gt;The essay key order - the rotation's frozen spine.&lt;/summary&gt;
public static readonly string[] EssayKeys =
{
    RestaurantTexts.KeyReviewEssayNews,
    RestaurantTexts.KeyReviewEssayDocu,
    RestaurantTexts.KeyReviewEssaySports,
    RestaurantTexts.KeyReviewEssayWuxia,
    RestaurantTexts.KeyReviewEssayGatha,
    RestaurantTexts.KeyReviewEssayStele,
    RestaurantTexts.KeyReviewEssayAcrostic,
    RestaurantTexts.KeyReviewEssayFoodcritic,
};
/// &lt;summary&gt;The genre label key order - index-matched to EssayKeys
/// (the wall line's [genre] tag - the visible signature).&lt;/summary&gt;
public static readonly string[] GenreNameKeys =
{
    RestaurantTexts.KeyGenreNews,
    RestaurantTexts.KeyGenreDocu,
    RestaurantTexts.KeyGenreSports,
    RestaurantTexts.KeyGenreWuxia,
    RestaurantTexts.KeyGenreGatha,
    RestaurantTexts.KeyGenreStele,
    RestaurantTexts.KeyGenreAcrostic,
    RestaurantTexts.KeyGenreFoodcritic,
};
/// &lt;summary&gt;The rotation cursor: a plain round-robin, negative-proof.
/// Consecutive collected reviews never share a genre (max run 1),
/// every genre stays reachable, the full cycle covers all eight.&lt;/summary&gt;
public static int GenreAt(int collectedIndex)
    =&gt; ((collectedIndex % GenreCount) + GenreCount) % GenreCount;
public static string EssayKeyAt(int collectedIndex)
    =&gt; EssayKeys[GenreAt(collectedIndex)];
/// &lt;summary&gt;The longest same-genre streak the rotation can produce
/// across any collected run (the anti-streak ceiling is 2 - the
/// round-robin lands at 1, never 3+ same-register screen spam).&lt;/summary&gt;
public static int MaxSameGenreRun(int totalCollected)
{
    if (totalCollected &lt;= 0) return 0;
    if (totalCollected == 1) return 1;
    return 1;
}
/// &lt;summary&gt;The grand-prix wall line for one collected review:
/// NO.{n} [genre] essay - the genre tag up front (the signature),
/// the essay text after (the register itself does the comedy).&lt;/summary&gt;
public static string WallLineAt(int collectedIndex)
{
    int g = GenreAt(collectedIndex);
    return string.Format(RestaurantTexts.EnText(RestaurantTexts.KeyReportWallEssayLine),
        collectedIndex + 1,
```

```
                RestaurantTexts.EnText(GenreNameKeys[g]),
                RestaurantTexts.EnText(EssayKeys[g]));
        }
        /// &lt;summary&gt;The whole collected essay list - the grand-prix long
        /// image body, one genre-signed line per collected review.&lt;/summary&gt;
        public static string WallBody(int totalCollected)
        {
            var sb = new StringBuilder();
            for (int i = 0; i &lt; totalCollected; i++)
            {
                if (i &gt; 0) sb.Append('\n');
                sb.Append(WallLineAt(i));
            }
            return sb.ToString();
        }
    }
}
// RestaurantReviewResponse.cs
// RestaurantReviewResponse.cs - the bad-review response system (G14-T014a,
// M05 the active decision face, SYS06 the word-correction wiring). The
// numeric pack section 4 modifier closed set owns every swing: a collected
// review lands -1, the reply-thanks response settles +1, the comp-meal
// response settles +2, and the boss-personal response rides the
// event-comedy band flip: +2 at the word_good band or hotter, -2 below -
// the two-way backfire tension (a grand personal reply reads as character
// when the house is hot, and as desperation to the review-literature flood
// when it is not). The comp books the budget line: one plain noodle at the
// frozen starter-dish menu price, refused clean when the wallet cannot
// cover it - never half-pay. Comping arms the freebie-drama-king chain
// mirror: the E03 face (cust_freebie_actor) lands same-tick as a pure
// telemetry beat (zero word - the closed set owns every swing; the stage
// wiring drives the visible comedy in T014b). One response per collected
// review, FIFO - each pending review is one &lt;=60s active-decision anchor
// (the pacing probe exposes the oldest pending age). Zero RNG, zero
// wall-clock, zero server (red line 12). RestaurantData and RestaurantServe
// stay untouched - the S1 frozen faces carry zero drift by construction;
// word settles only through the serving engine so the 0-100 clamp law
// owns every swing.
using System.Collections.Generic;
using System.Text;
namespace Sy.CrazyRestaurant
{
    public enum RestaurantResponseKind { ReplyThanks = 0, CompMeal = 1, BossPersonally = 2 }
    /// &lt;summary&gt;One logged response beat: the answered review, the chosen
    /// slot, the settled word swing and the budget line.&lt;/summary&gt;
    public sealed class RestaurantResponseBeat
    {
        public double t;
        public double reviewAt;            // the answered review's arrival time (FIFO)
        public RestaurantResponseKind kind;
```

```
    public int wordSwing;              // +1 / +2 / the band-flipped +/-2
    public int costHc;                 // 0 / the frozen comp price / 0
    public bool chainMirrored;         // the freebie-drama-king mirror (comp only)
}
public sealed class RestaurantReviewResponse
{
    public const int ResponseSlotCount = 3;
    public const int ReplyWordSwing = 1;     // pack section 4: reply-thanks +1
    public const int CompWordSwing = 2;      // pack section 4: comp +2
    public const int BossWordSwing = 2;      // the event-comedy band flip magnitude
    public const int BossFlipFloorWord = 50; // the word_good band low bound - the flip anchor
    public const string FreebieChainFunnelId = "evt_freebie_chain";
    public const string ResponseFunnelId = "evt_review_response";
    public const string PendingFunnelId = "evt_review_pending";
    readonly RestaurantServe _serve;
    readonly List&lt;double&gt; _pending = new List&lt;double&gt;();
    readonly List&lt;RestaurantResponseBeat&gt; _beats = new List&lt;RestaurantResponseBeat&gt;();
    readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
    int _spentHc;
    int _refusedEmpty;
    int _refusedBudget;
    public RestaurantReviewResponse(RestaurantServe serve) { _serve = serve; }
    // ---- live faces ----
    public RestaurantServe Serve =&gt; _serve;
    public int PendingCount =&gt; _pending.Count;
    public int RespondedCount =&gt; _beats.Count;
    public int SpentHc =&gt; _spentHc;
    public int RefusedEmptyCount =&gt; _refusedEmpty;
    public int RefusedBudgetCount =&gt; _refusedBudget;
    public IList&lt;RestaurantResponseBeat&gt; Beats =&gt; _beats;
    public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
    /// &lt;summary&gt;The comp books one plain noodle: the always-on starter
    /// dish's frozen menu price (the humble comp bowl).&lt;/summary&gt;
    public static int CompMealCostHc =&gt; RestaurantData.Recipes()[0].dishPriceHc;
    /// &lt;summary&gt;The budget pre-check face: the reply and the boss slot
    /// ride free, the comp rides the frozen starter-dish price.&lt;/summary&gt;
    public static int CostOf(RestaurantResponseKind kind)
        =&gt; kind == RestaurantResponseKind.CompMeal ? CompMealCostHc : 0;
    /// &lt;summary&gt;The word swing a slot settles (the boss face reads the
    /// live band - the two-way backfire): +1 reply / +2 comp /
    /// band-flipped +/-2 boss.&lt;/summary&gt;
    public int SwingOf(RestaurantResponseKind kind)
    {
        if (kind == RestaurantResponseKind.ReplyThanks) return ReplyWordSwing;
        if (kind == RestaurantResponseKind.CompMeal) return CompWordSwing;
        return _serve.Word &gt;= BossFlipFloorWord ? BossWordSwing : -BossWordSwing;
    }
    /// &lt;summary&gt;Review intake: a collected review (the E02 beat) joins
    /// the pending queue - one decision anchor per review.&lt;/summary&gt;
    public void NoteReviewCollected(double t)
```

```
{
    _pending.Add(t);
    _funnel.Add(PendingFunnelId + "@" + t.ToString("0.00"));
}
/// &lt;summary&gt;The oldest pending review's arrival time (-1 when the
/// queue is empty).&lt;/summary&gt;
public double OldestPendingAt =&gt; _pending.Count &gt; 0 ? _pending[0] : -1.0;
/// &lt;summary&gt;The decision-anchor pacing probe: seconds the oldest
/// pending review has waited at 'now' (-1 when the queue is empty)
/// - the &lt;=60s active-decision cadence face (the stage wiring
/// drives the beat).&lt;/summary&gt;
public double OldestPendingAge(double now)
    =&gt; _pending.Count &gt; 0 ? now - _pending[0] : -1.0;
/// &lt;summary&gt;The active-decision transaction: answers the oldest
/// pending review with the chosen slot. The budget check runs
/// before the settle (insufficient gold refuses clean - never
/// half-pays), the word swing settles through the serving engine
/// (the 0-100 clamp law owns it), the comp books the spend ledger
/// and arms the freebie-drama-king chain mirror same-tick.&lt;/summary&gt;
public bool Respond(double t, RestaurantResponseKind kind, int goldAvailableHc)
{
    if (_pending.Count == 0) { _refusedEmpty++; return false; }
    int cost = CostOf(kind);
    if (goldAvailableHc &lt; cost) { _refusedBudget++; return false; }
    double reviewAt = _pending[0];
    _pending.RemoveAt(0);
    int swing = SwingOf(kind);
    _serve.SetWord(_serve.Word + swing);
    _spentHc += cost;
    var beat = new RestaurantResponseBeat
    {
        t = t,
        reviewAt = reviewAt,
        kind = kind,
        wordSwing = swing,
        costHc = cost,
        chainMirrored = false
    };
    if (kind == RestaurantResponseKind.CompMeal)
    {
        beat.chainMirrored = true;
        _funnel.Add(FreebieChainFunnelId + "@" + t.ToString("0.00"));
    }
    _beats.Add(beat);
    _funnel.Add(ResponseFunnelId + "@" + t.ToString("0.00"));
    return true;
}
public int CountOf(RestaurantResponseKind kind)
{
    int n = 0;
```

```
            for (int i = 0; i &lt; _beats.Count; i++)
                if (_beats[i].kind == kind) n++;
            return n;
        }
        public int ChainMirrorCount
        {
            get
            {
                int n = 0;
                for (int i = 0; i &lt; _beats.Count; i++)
                    if (_beats[i].chainMirrored) n++;
                return n;
            }
        }
        /// &lt;summary&gt;Deterministic fingerprint - the double-run byte-parity
        /// face (zero-RNG family): identical drives land on identical bytes.&lt;/summary&gt;
        public string ResponseSig()
        {
            var sb = new StringBuilder();
            sb.Append("pending=").Append(_pending.Count)
              .Append("|responded=").Append(_beats.Count)
              .Append("|spent=").Append(_spentHc)
              .Append("|refused=").Append(_refusedEmpty).Append(',').Append(_refusedBudget)
              .Append("|funnel=").Append(_funnel.Count);
            for (int i = 0; i &lt; _beats.Count; i++)
            {
                var b = _beats[i];
                sb.Append(";").Append((int)b.kind).Append('~')
                  .Append(b.reviewAt.ToString("0.###")).Append('&gt;')
                  .Append(b.t.ToString("0.###"))
                  .Append("w").Append(b.wordSwing)
                  .Append("c").Append(b.costHc)
                  .Append(b.chainMirrored ? "#chain" : "");
            }
            return sb.ToString();
        }
    }
}
// RestaurantServe.cs
// RestaurantServe.cs - the serving income engine (G14-T005, SYS01/SYS02).
// Pure C# zero-RNG simulation: an open stall auto-cooks the assigned recipe
// (the wok-to-counter three-beat serve), a finished dish auto-sells the
// moment the counter is open (the idle-genre "no dead dish" law), the
// passive stove output pays the baseline flow, recipe R&amp;D pays its prep
// cost, runs the frozen research window and lands the researched dish on
// the menu with a one-time demo bowl sold at its frozen price (the
// coin-burst beat), and the booked income per cap window respects the rank
// ceiling. Preset stove slots open on the linear cost law (pick WHAT opens,
// never WHERE). Wallet math is exact by construction: gold == start - stove
// costs - prep costs - research cost + booked ledger.
```

```
using System.Collections.Generic;
namespace Sy.CrazyRestaurant
{
    /// &lt;summary&gt;Per-line booked income ledger (integer hundredths).&lt;/summary&gt;
    public sealed class ServeLedger
    {
        public int soldHc;       // dish sales booked (serves + the R&amp;D demo bowl)
        public int stoveHc;      // passive stove output booked
        public int spilledHc;    // income above the per-window cap, turned away
        public int TotalHc =&gt; soldHc + stoveHc;
    }
    public sealed class RestaurantServe
    {
        // ---- v1 engine wiring (sweep-tunable; the frozen economy laws live
        // in RestaurantData - these carry only the opening feel) ----
        public const int StartGoldHc = 10000;             // 100.00 gold: covers the R1 first stove + the first preps +
        public const int StartWord = 50;                  // the par band opens the diner's life
        public const double FirstBowlDelaySeconds = 0.2;  // the wok warms up a fifth of a second after the hook
        public const double ResearchStartDelaySeconds = 0.2;  // the R&amp;D beat starts a fifth of a second after the f
        public const double ResearchWindowSeconds = 5.5;  // the opening research window (the demo bowl lands &lt;= 8s f
        public const int CapWindowSeconds = 10;           // the income-ceiling reconciliation window
        RankRow _rank;                  // the economy row (T008b: PromoteTo swaps it up the ladder)
        double[] _timers;               // remaining cook seconds per stove, &lt;= 0 with a dish = ready awaiting the co
        string[] _dishes;               // recipe id per stove, "" = idle
        bool[] _ready;                  // a finished dish stands ready until the counter opens
        readonly List&lt;string&gt; _menu = new List&lt;string&gt;();
        readonly ServeLedger _ledger = new ServeLedger();
        double _stoveAcc, _soldAcc;      // fractional hc accumulators (deterministic double math)
        double _windowAcc;                // booked income within the current cap window
        double _maxWindowAcc;             // worst cap-window booked flow (the ceiling instrument)
        double _secondStart;              // the current cap window start (sim seconds)
        int _stoveSpentHc;
        int _prepSpentHc;
        int _researchSpentHc;
        int _facilitySpentHc;                    // the facility build spends (T013c, zero = the frozen face)
        int _responseSpentHc;                    // the comp-response spends (T014b, zero = the frozen face)
        int _openStoves;
        int _word;
        int _servedTotal;
        int _researchDone;
        int _lastChoice = -1;
        int _lastSaleHc;
        double _t;
        // the chef-card factor faces (T006a): zero defaults keep the T005
        // face bit-identical - every application branches on its factor so
        // the zero path runs the original expression unchanged.
        int _cardAlbumPct;      // completed-volume income bonus (the album law)
        int _cardPricePct;      // the cashier's mental-math price lift
        int _cardServePct;      // the zen prep's half-beat cost (negative = slower cook)
        int _cardResearchPct;   // the brave taster's R&amp;D window speedup
```

```
int _facilityHallCapBp = 10000;        // the dining-hall queue-capacity ceiling face (T013c, 10000 = the frozen
int _facilityRoomTicketBp = 10000;     // the private-room per-head ticket face (T013c)
int _facilityKitchenOutputBp = 10000;  // the central-kitchen whole-line output face (T013c)
int _boostResearchBp = 10000;         // the research-window boost factor (T015b, 10000 = the frozen face)
// the R&amp;D face (one research at a time)
double _researchTimer = -1;      // &lt; 0 = no research running
int _researchRecipeIndex = -1;
double _researchDoneAt = -1;
double _firstCoinAt = -1;
public RestaurantServe(int rank = 1)
{
    var ranks = RestaurantData.Ranks();
    int i = rank - 1;
    if (i &lt; 0 || i &gt;= ranks.Length) i = 0;
    _rank = ranks[i];
    _timers = new double[_rank.stoves];
    _dishes = new string[_rank.stoves];
    _ready = new bool[_rank.stoves];
    _word = StartWord;
    _menu.Add(RestaurantData.Recipes()[0].id);   // the street stall opens with the plain noodle
}
// ---- live state faces ----
public double SimTime =&gt; _t;
public int GoldHc =&gt; StartGoldHc - _stoveSpentHc - _prepSpentHc - _researchSpentHc - _facilitySpentHc - _resp
public int Word =&gt; _word;
public int OpenStoves =&gt; _openStoves;
public int ServedTotal =&gt; _servedTotal;
public int ResearchDone =&gt; _researchDone;
public double ResearchDoneAt =&gt; _researchDoneAt;
public double FirstCoinAt =&gt; _firstCoinAt;
public ServeLedger Ledger =&gt; _ledger;
public double MaxBookedPerWindow =&gt; _maxWindowAcc;
public RankRow Rank =&gt; _rank;
public int LastChoice =&gt; _lastChoice;
public int StovesTotal =&gt; _timers.Length;
public IReadOnlyList&lt;string&gt; Menu =&gt; _menu;
public bool StallOpen =&gt; _stallOpen;
public int LastSaleHc =&gt; _lastSaleHc;   // the price the last sale booked (the coin-jump face)
bool _stallOpen;
/// &lt;summary&gt;True when the stove holds a finished dish awaiting the
/// counter (the steaming-window face).&lt;/summary&gt;
public bool IsReady(int stoveIndex)
    =&gt; stoveIndex &gt;= 0 &amp;&amp; stoveIndex &lt; _timers.Length &amp;&amp; _ready[stoveIndex];
/// &lt;summary&gt;Remaining cook seconds of a stove (-1 when idle).&lt;/summary&gt;
public double RemainingSeconds(int stoveIndex)
    =&gt; stoveIndex &gt;= 0 &amp;&amp; stoveIndex &lt; _timers.Length &amp;&amp; !string.IsNullOrEmpty(_dishes[
        ? (_timers[stoveIndex] &gt; 0 ? _timers[stoveIndex] : 0) : -1;
/// &lt;summary&gt;Word-of-mouth setter with the frozen 0-100 clamp.&lt;/summary&gt;
public void SetWord(int value)
{
```

```
    _word = value &lt; RestaurantData.WordMin ? RestaurantData.WordMin
        : value &gt; RestaurantData.WordMax ? RestaurantData.WordMax : value;
}
/// &lt;summary&gt;Chef-card factor injection (T006a): the card system
/// resolves its owned passives here (album bonus / price / serve
/// speed / research speed). Zero defaults = the T005 face unchanged
/// (no application branch ever fires on a zero factor).&lt;/summary&gt;
public void SetCardFactors(int albumPct, int pricePct, int servePct, int researchPct)
{
    _cardAlbumPct = albumPct;
    _cardPricePct = pricePct;
    _cardServePct = servePct;
    _cardResearchPct = researchPct;
}
public WordBand Band =&gt; RestaurantData.BandFor(_word);
/// &lt;summary&gt;The facility wallet line (T013c): the build-menu rungs
/// book through the same wallet - the stage reconciles the facility
/// ledger onto this line tick-for-tick. Zero bookings keep the
/// frozen face byte-identical.&lt;/summary&gt;
public void BookFacilitySpend(int hc)
{
    if (hc &gt; 0) _facilitySpentHc += hc;
}
public int FacilitySpentHc =&gt; _facilitySpentHc;
/// &lt;summary&gt;Books a comp-response spend (T014b): the review-answer
/// comp books the same tick the response settles - the wallet and the
/// response ledger reconcile tick-for-tick, a refused answer never
/// half-pays. Zero (or junk) calls keep the frozen face
/// byte-identical.&lt;/summary&gt;
public void BookResponseSpend(int hc)
{
    if (hc &gt; 0) _responseSpentHc += hc;
}
public int ResponseSpentHc =&gt; _responseSpentHc;
/// &lt;summary&gt;The facility factors (T013c): the dining hall lifts the
/// queue-capacity income ceiling, the private room lifts the
/// per-head ticket on every dish sale, the central kitchen lifts the
/// whole-line stove output - each on its own bp face. Zero (or
/// junk, clamped) calls keep the frozen T005 face byte-identical:
/// every application branches on its factor so the neutral path
/// runs the original expression unchanged.&lt;/summary&gt;
public void SetFacilityFactors(int hallCapBp, int roomTicketBp, int kitchenOutputBp)
{
    _facilityHallCapBp = hallCapBp &lt;= 0 ? 10000 : hallCapBp;
    _facilityRoomTicketBp = roomTicketBp &lt;= 0 ? 10000 : roomTicketBp;
    _facilityKitchenOutputBp = kitchenOutputBp &lt;= 0 ? 10000 : kitchenOutputBp;
}
public int FacilityHallCapBp =&gt; _facilityHallCapBp;
public int FacilityRoomTicketBp =&gt; _facilityRoomTicketBp;
public int FacilityKitchenOutputBp =&gt; _facilityKitchenOutputBp;
```

```
/// &lt;summary&gt;The research-window boost factor (T015b): the completed
/// boost view lands x2 on the current research drain - the window is
/// the research, the completion tick clears the factor back to
/// neutral (a no-op on every unboosted beat, so the frozen face
/// stays byte-identical). Junk (or zero) calls clamp to
/// neutral.&lt;/summary&gt;
public void SetBoostFactorBp(int bp)
{
    _boostResearchBp = bp &lt;= 0 ? 10000 : bp;
}
public int BoostFactorBp =&gt; _boostResearchBp;
/// &lt;summary&gt;Rank move (T008b): swaps the economy row up the ladder
/// and extends the stove arrays (the new stoves arrive idle, the
/// stall keeps its open face). The promotion settle lands here so
/// the ladder rank and the economy row can never split; zero calls
/// keep the frozen T005 face byte-identical.&lt;/summary&gt;
public void PromoteTo(int rank)
{
    var ranks = RestaurantData.Ranks();
    int i = rank - 1;
    if (i &lt; 0 || i &gt;= ranks.Length) return;   // defensive: the ladder owns the moves
    var row = ranks[i];
    if (row.rank &lt;= _rank.rank) return;       // the ladder only climbs
    var timers = new double[row.stoves];
    var dishes = new string[row.stoves];
    var ready = new bool[row.stoves];
    for (int k = 0; k &lt; _timers.Length &amp;&amp; k &lt; timers.Length; k++)
    {
        timers[k] = _timers[k];
        dishes[k] = _dishes[k];
        ready[k] = _ready[k];
    }
    _rank = row;
    _timers = timers;
    _dishes = dishes;
    _ready = ready;
}
/// &lt;summary&gt;Opens the counter: the stall decision (the serve beat
/// unlocks - a finished dish can finally sell).&lt;/summary&gt;
public bool OpenStall(int choice)
{
    if (_stallOpen) return false;
    _stallOpen = true;
    _lastChoice = choice;
    return true;
}
/// &lt;summary&gt;Opens the next preset stove slot (pick WHAT opens, never
/// WHERE - the schema law). Fails when the wallet or the rank slot
/// chain is exhausted.&lt;/summary&gt;
public bool OpenNextStove(int choice)
```

```
{
    int next = _openStoves + 1;
    if (next &gt; _rank.stoves) return false;
    int cost = RestaurantData.StoveCost(_rank, next);
    if (GoldHc &lt; cost) return false;
    _stoveSpentHc += cost;
    _openStoves = next;
    _lastChoice = choice;
    return true;
}
/// &lt;summary&gt;Starts a dish on an open, idle stove: pays the prep cost
/// and starts the frozen cook countdown.&lt;/summary&gt;
public bool StartDish(int stoveIndex, int recipeIndex)
{
    if (stoveIndex &lt; 0 || stoveIndex &gt;= _openStoves) return false;
    if (!string.IsNullOrEmpty(_dishes[stoveIndex]) || _ready[stoveIndex]) return false;
    var recipes = RestaurantData.Recipes();
    if (recipeIndex &lt; 0 || recipeIndex &gt;= recipes.Length) return false;
    if (!_menu.Contains(recipes[recipeIndex].id)) return false;   // only researched recipes cook
    int cost = recipes[recipeIndex].prepCost;
    if (GoldHc &lt; cost) return false;
    _prepSpentHc += cost;
    _dishes[stoveIndex] = recipes[recipeIndex].id;
    _timers[stoveIndex] = recipes[recipeIndex].cookSeconds;
    return true;
}
/// &lt;summary&gt;Starts the R&amp;D beat on a recipe not yet on the menu:
/// pays the research cost (its frozen prep row - a batch's worth of
/// experimenting) and runs the research window. On completion the
/// recipe lands on the menu and the demo bowl sells once at its
/// frozen price (the coin-burst beat).&lt;/summary&gt;
public bool TryStartResearch(int recipeIndex)
{
    var recipes = RestaurantData.Recipes();
    if (recipeIndex &lt; 0 || recipeIndex &gt;= recipes.Length) return false;
    if (_menu.Contains(recipes[recipeIndex].id)) return false;    // already researched
    if (_researchTimer &gt;= 0) return false;                         // one research at a time
    int cost = recipes[recipeIndex].prepCost;
    if (GoldHc &lt; cost) return false;
    _researchSpentHc += cost;
    _researchRecipeIndex = recipeIndex;
    _researchTimer = _cardResearchPct == 0
        ? ResearchWindowSeconds
        : ResearchWindowSeconds * 100.0 / (100.0 + _cardResearchPct);   // the taster shrinks the window
    return true;
}
public bool ResearchRunning =&gt; _researchTimer &gt;= 0;
public int ResearchRecipeIndex =&gt; _researchRecipeIndex;
/// &lt;summary&gt;Dark-research wallet face (T009a): pays a dark-dish
/// research cost on the wallet's spend side. Additive law - the
```

```
/// exact-wallet identity keeps holding and the fingerprint format
/// never moves; a refused tap (empty wallet) leaves zero trace.&lt;/summary&gt;
public bool TrySpendDarkResearch(int costHc)
{
    if (costHc &lt;= 0) return false;
    if (GoldHc &lt; costHc) return false;
    _researchSpentHc += costHc;
    return true;
}
/// &lt;summary&gt;Offline doubler landing (T010): a completed doubler
/// claim books the span's whole earned income a second time
/// straight onto the sold ledger - uncapped by design (the x2 of
/// already-capped income is a payout, not new window flow). The
/// fractional accumulator rides along so the floor law stays
/// exact for every later booking. Additive law: the exact-wallet
/// identity keeps holding and the fingerprint format never moves;
/// zero calls keep the frozen T005 face byte-identical.&lt;/summary&gt;
public void BookOfflineDouble(int hc)
{
    if (hc &lt;= 0) return;
    _soldAcc += hc;
    _ledger.soldHc += hc;
}
/// &lt;summary&gt;Advances the sim in cap-window slices so the booked flow
/// per window reconciles exactly: passive stove output, cook
/// countdowns with auto-serve on completion (the counter must be
/// open), and the research window. dt in seconds, any positive
/// value.&lt;/summary&gt;
public void Advance(double dt)
{
    if (dt &lt;= 0) return;
    double remaining = dt;
    while (remaining &gt; 0)
    {
        double boundary = _secondStart + CapWindowSeconds;
        double seg = System.Math.Min(remaining, boundary - _t);
        if (seg &lt;= 0) seg = remaining;   // defensive: never stall the clock
        _t += seg;
        remaining -= seg;
        if (_openStoves &gt; 0)
        {
            double stoveRateHc = _cardAlbumPct == 0
                ? _rank.stoveOutputHc
                : _rank.stoveOutputHc * (100.0 + _cardAlbumPct) / 100.0;
            if (_facilityKitchenOutputBp != 10000)
                stoveRateHc = stoveRateHc * _facilityKitchenOutputBp / 10000.0;   // the central-kitchen whole-l
            Book(ref _stoveAcc, ref _ledger.stoveHc, stoveRateHc * (double)_openStoves * seg);
        }
        for (int i = 0; i &lt; _timers.Length; i++)
        {
```

```
        if (i &gt;= _openStoves) break;   // only open stoves cook
        if (string.IsNullOrEmpty(_dishes[i])) continue;
        if (!_ready[i])
        {
            double cookSeg = _cardServePct == 0 ? seg : seg * (100.0 + _cardServePct) / 100.0;
            _timers[i] -= cookSeg;
            if (_timers[i] &lt; 1e-6) _timers[i] = 0;   // snap float residue: done is done
            if (_timers[i] &lt;= 0) _ready[i] = true;
        }
        if (_ready[i] &amp;&amp; _stallOpen)
        {
            // the three-beat serve: the dish lands on the counter and
            // the coin fires on the same tick (auto-serve, no dead dish)
            var recipes = RestaurantData.Recipes();
            foreach (var r in recipes)
            {
                if (r.id != _dishes[i]) continue;
                double saleHc = (_cardPricePct == 0 &amp;&amp; _cardAlbumPct == 0)
                    ? r.dishPriceHc
                    : r.dishPriceHc * (100.0 + _cardPricePct + _cardAlbumPct) / 100.0;
                if (_facilityRoomTicketBp != 10000)
                    saleHc = saleHc * _facilityRoomTicketBp / 10000.0;   // the private-room per-head ticket
                Book(ref _soldAcc, ref _ledger.soldHc, saleHc);
                _lastSaleHc = (int)System.Math.Floor(saleHc + 1e-9);
                if (_firstCoinAt &lt; 0) _firstCoinAt = _t;
                _servedTotal++;
                break;
            }
            _dishes[i] = "";
            _ready[i] = false;
            _timers[i] = 0;
        }
    }
    if (_researchTimer &gt;= 0)
    {
        if (_boostResearchBp != 10000)
            _researchTimer -= seg * _boostResearchBp / 10000.0;   // the research-window boost books its win
        else
            _researchTimer -= seg;
        if (_researchTimer &lt; 1e-6) _researchTimer = 0;
        if (_researchTimer &lt;= 0)
        {
            // the research lands: the recipe joins the menu and the
            // demo bowl sells once at its frozen price (coin burst)
            var recipes = RestaurantData.Recipes();
            if (_researchRecipeIndex &gt;= 0 &amp;&amp; _researchRecipeIndex &lt; recipes.Length)
            {
                _menu.Add(recipes[_researchRecipeIndex].id);
                double demoHc = (_cardPricePct == 0 &amp;&amp; _cardAlbumPct == 0)
                    ? recipes[_researchRecipeIndex].dishPriceHc
```

```
                        : recipes[_researchRecipeIndex].dishPriceHc * (100.0 + _cardPricePct + _cardAlbumPct) /
                    if (_facilityRoomTicketBp != 10000)
                        demoHc = demoHc * _facilityRoomTicketBp / 10000.0;   // the private-room per-head ticket
                    Book(ref _soldAcc, ref _ledger.soldHc, demoHc);
                    _lastSaleHc = (int)System.Math.Floor(demoHc + 1e-9);
                }
                _researchDone++;
                _researchDoneAt = _t;
                _researchTimer = -1;
                _researchRecipeIndex = -1;
                _boostResearchBp = 10000;   // the window ends with the research: the completion tick clears the
            }
        }
        if (_t &gt;= boundary)
        {
            if (_windowAcc &gt; _maxWindowAcc) _maxWindowAcc = _windowAcc;
            _secondStart = boundary;
            _windowAcc = 0;
        }
    }
}
void Book(ref double acc, ref int booked, double amountHc)
{
    double ceiling = _rank.incomeCapHc * (double)CapWindowSeconds;
    if (_facilityHallCapBp != 10000)
        ceiling = ceiling * _facilityHallCapBp / 10000.0;   // the dining-hall queue capacity lifts the ceiling
    double headroom = ceiling - _windowAcc;
    double bookedPart = amountHc;
    if (headroom &lt; bookedPart)
    {
        bookedPart = headroom &gt; 0 ? headroom : 0;
        _ledger.spilledHc += (int)System.Math.Floor(amountHc - bookedPart);
    }
    _windowAcc += bookedPart;
    acc += bookedPart;
    int whole = (int)System.Math.Floor(acc);
    if (whole &gt; booked) booked = whole;   // ledger is the wallet source: exact by construction
}
/// &lt;summary&gt;Canonical fingerprint: the zero-RNG parity instrument
/// (same script = same bytes).&lt;/summary&gt;
public string Fingerprint()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("t=").Append(_t.ToString("0.###"));
    sb.Append("|word=").Append(_word);
    sb.Append("|gold=").Append(GoldHc);
    sb.Append("|stoves=").Append(_openStoves).Append('/').Append(_rank.stoves);
    sb.Append("|choice=").Append(_lastChoice);
    sb.Append("|served=").Append(_servedTotal);
    sb.Append("|research=").Append(_researchDone);
```

```
            sb.Append("|stovelines=");
            for (int i = 0; i &lt; _timers.Length; i++)
                sb.Append(_dishes[i]).Append('@').Append(_timers[i].ToString("0.###"))
                  .Append(_ready[i] ? "!" : "").Append(',');
            sb.Append("|led=").Append(_ledger.soldHc).Append(',').Append(_ledger.stoveHc)
              .Append(',').Append(_ledger.spilledHc);
            sb.Append("|maxwin=").Append(_maxWindowAcc.ToString("0.###"));
            sb.Append("|menu=");
            foreach (var m in _menu) sb.Append(m).Append(',');
            sb.Append("|cards=").Append(_cardAlbumPct).Append(',').Append(_cardPricePct)
              .Append(',').Append(_cardServePct).Append(',').Append(_cardResearchPct);
            return sb.ToString();
        }
    }
}
// RestaurantStage.cs
// RestaurantStage.cs - the opening beat driver (G14-T005).
// Drives RestaurantServe through the frozen red-line-11 opening script and
// stamps every promise on the sim clock: the street-food steam hook owns
// t0, the stall decision lands one second in (live right after the hook,
// zero login wall - one tap opens the first stove AND the counter), the
// first bowl starts a fifth of a second later and the first coin fires the
// visual+sfx+number triple on one tick inside the 1.5s anchor from the
// decision, the first recipe research runs its frozen window and lands the
// demo-bowl coin burst inside the 8s anchor, then the blaze-chef event card
// (the first laugh) and the midnight-guest preview land inside the 10s
// window (T007c: the wired RestaurantEventEngine's E01 beat drives that
// triple - the frozen face stays byte-identical, OpeningSig carries no
// engine faces). Pure C# zero-RNG: the same script always fingerprints
// byte-identical (Redline11Check consumes this face).
using System.Collections.Generic;
namespace Sy.CrazyRestaurant
{
    public enum RestaurantPhase
    {
        Hook,           // the steam panorama + the stall caption
        StallOpen,      // the stall decision made, counter live
        FirstCoin,      // the first bowl sold, coins burst
        FirstResearch,  // the first recipe researched, the demo bowl sold
    }
    public sealed class RestaurantStage
    {
        // the frozen opening script (red-line-11 beat anchors)
        public const double FirstStallDecisionAt = 1.0;   // T+1s: the stall decision (live right after the hook)
        public const double FirstEventCardAt = 9.5;        // T+9.5s: the blaze-bangs card + the midnight preview
        readonly RestaurantServe _serve;
        readonly RestaurantEventEngine _events;   // T007c: the diner event engine rides the same sim
        readonly ChefCards _cards;                // T008b: the dish-album dimension (the UI layer drives the pulls)
        readonly RestaurantRankEngine _rank;      // T008b: the quality-audit ladder rides the same sim
        readonly RestaurantFacility _facility;    // T013c: the facility board rides the stage (zero taps = the frozen f
```

```
readonly RestaurantReviewResponse _reviewResp;  // T014b: the bad-review response engine rides the stage
int _reviewsSeen;                          // T014b: the E02 firings already landed in the pending queue
readonly List&lt;(string id, double t)&gt; _stamps = new List&lt;(string, double)&gt;();
double _t;
bool _stallOpen;
double _stallOpenedAt = -1;
bool _bowlIn;
bool _coinDone;
double _coinAt = -1;
bool _researchDone;
bool _eventDone;
string _firstEventCaption = "";
int _coinJumpHc;
RestaurantStage(RestaurantServe serve, int startRank)
{
    _serve = serve;
    _events = new RestaurantEventEngine(serve, startRank);   // T007c: the engine rides the serve sim, scoped at
    _cards = new ChefCards(serve, null);              // T008b: fresh empty albums (the UI layer wires the ad po
    _rank = new RestaurantRankEngine(serve, _cards, startRank);   // T008b: the ladder starts at the arrival ran
    _facility = new RestaurantFacility();                     // T013c: fresh empty board (nothing built)
    _reviewResp = new RestaurantReviewResponse(serve);        // T014b: the response engine rides the serve sim
    _reviewsSeen = 0;
}
/// &lt;summary&gt;The fresh boss: one preset stove slot waiting, the plain
/// noodle on the starter menu, the par-band word of a new diner.&lt;/summary&gt;
public static RestaurantStage NewBoss()
    =&gt; new RestaurantStage(new RestaurantServe(1), 1);
/// &lt;summary&gt;The seasoned boss (T013c): a stage arrived at rank R -
/// the staged facility floors unlock off the arrival rank the way
/// the stove rows do (the wiring lane for the R3/R4/R5 build
/// menu; the fresh boss still opens at R1).&lt;/summary&gt;
public static RestaurantStage AtRank(int rank)
    =&gt; new RestaurantStage(new RestaurantServe(rank), rank);
// ---- live faces ----
public double SimTime =&gt; _t;
public RestaurantPhase Phase
{
    get
    {
        if (_researchDone) return RestaurantPhase.FirstResearch;
        if (_coinDone) return RestaurantPhase.FirstCoin;
        if (_stallOpen) return RestaurantPhase.StallOpen;
        return RestaurantPhase.Hook;
    }
}
public RestaurantServe Serve =&gt; _serve;
public RestaurantEventEngine Events =&gt; _events;   // T007c wiring face
public ChefCards Cards =&gt; _cards;                 // T008b wiring face
public RestaurantRankEngine RankEngine =&gt; _rank;   // T008b wiring face
public RestaurantFacility Facility =&gt; _facility;   // T013c wiring face (the build board)
```

```
public RestaurantReviewResponse ReviewResponse =&gt; _reviewResp;   // T014b wiring face (the pending queue + th
/// &lt;summary&gt;The &lt;=60s decision-anchor cadence face (T014b): the
/// oldest pending review's wait in sim seconds (-1 when no review
/// awaits) - one active-decision anchor per collected review.&lt;/summary&gt;
public double DecisionAnchorAge =&gt; _reviewResp.OldestPendingAge(_t);
/// &lt;summary&gt;The decision window (T014b): a pending review inside its
/// fresh &lt;=60s anchor band - the UI surfaces the beat while the
/// window is open.&lt;/summary&gt;
public bool DecisionWindowOpen =&gt; _reviewResp.PendingCount &gt; 0 &amp;&amp; _reviewResp.OldestPendingAge(_t)
public bool ModalActive =&gt; false;              // zero login wall, zero blocking modal (red line 11)
public bool CanDecide =&gt; _stallOpen == false; // the stall decision is live right after the hook
public IReadOnlyList&lt;(string id, double t)&gt; Stamps =&gt; _stamps;
/// &lt;summary&gt;First stamp time of an event id (-1 when never fired).&lt;/summary&gt;
public double TimeOf(string id)
{
    for (int i = 0; i &lt; _stamps.Count; i++)
        if (_stamps[i].id == id) return _stamps[i].t;
    return -1;
}
void Stamp(string id)
{
    _stamps.Add((id, _t));
}
/// &lt;summary&gt;Advances the opening clock: drives the serve engine and
/// fires the scheduled beats (first bowl, first coin, first research,
/// event cards) on the frozen script.&lt;/summary&gt;
public void Advance(double dt)
{
    if (dt &lt;= 0) return;
    if (TimeOf("evt_restaurant_hook") &lt; 0)
        Stamp("evt_restaurant_hook");   // the hook owns t0 (steam panorama + street-food caption)
    _t += dt;
    _serve.Advance(dt);
    _events.Advance(dt);   // T007c: the engine rides the same sim clock
    // T014b: every E02 review essay lands in the pending queue - one
    // decision anchor per collected review (pure intake, zero stamps
    // and zero wallet: the frozen face stays byte-identical, the
    // response engine owns its own funnel)
    int collectedReviews = _events.ReviewsCollected;
    while (_reviewsSeen &lt; collectedReviews)
    {
        _reviewResp.NoteReviewCollected(_t);
        _reviewsSeen++;
    }
    // the first bowl starts a fifth of a second after the stall opens
    if (_stallOpen &amp;&amp; !_bowlIn &amp;&amp; _t &gt;= _stallOpenedAt + RestaurantServe.FirstBowlDelaySecond
    {
        if (_serve.StartDish(0, 0))
        {
            Stamp("evt_first_bowl");
```

```
            Stamp("bowl_visual");   // the wok smoke face
            Stamp("bowl_sfx");      // the signature sizzle trigger
            Stamp("prep_number");   // the gold counter tick (prep paid)
            _bowlIn = true;
        }
    }
    // the first coin: the finished bowl auto-sells and the coin fires
    // visual+sfx+number on one tick (the serve triple)
    if (_bowlIn &amp;&amp; !_coinDone &amp;&amp; _serve.FirstCoinAt &gt;= 0)
    {
        _coinJumpHc = _serve.LastSaleHc;   // the price the first sale booked
        Stamp("evt_first_coin");
        Stamp("coin_visual");   // the coin popup face
        Stamp("coin_sfx");     // the signature coin chime
        Stamp("coin_number");  // the gold counter jump
        _coinDone = true;
        _coinAt = _serve.FirstCoinAt;
    }
    // the first research: starts a fifth of a second after the first
    // coin (the scripted R&amp;D beat on the big-plate noodle)
    if (_coinDone &amp;&amp; !_researchDone &amp;&amp; _serve.ResearchRecipeIndex &lt; 0
        &amp;&amp; _t &gt;= _coinAt + RestaurantServe.ResearchStartDelaySeconds)
    {
        if (_serve.TryStartResearch(1))
            Stamp("evt_research_started");
    }
    // the research lands: the recipe joins the menu and the demo bowl
    // fires the coin-burst triple on one tick (the idle feel beat)
    if (!_researchDone &amp;&amp; _serve.ResearchDoneAt &gt;= 0)
    {
        Stamp("evt_first_research");
        Stamp("burst_visual");   // the new dish face
        Stamp("burst_sfx");     // the wok-toss chime
        Stamp("burst_number");  // the gold counter jump
        _researchDone = true;
    }
    // the first event cards: the wired engine's E01 beat drives the
    // stamps (the blaze chef's bangs catch fire - the first laugh -
    // with the review-intel preview chained on the same tick); the
    // stamps land on the stage tick that observes the beat, the frozen
    // T005 face byte-identical (zero drift), OpeningSig carries no
    // engine faces
    if (_stallOpen &amp;&amp; !_eventDone &amp;&amp; _events.CountCards("evt_blaze_bangs") &gt; 0)
    {
        Stamp("evt_blaze_bangs");
        Stamp("evt_first_laugh");
        Stamp("evt_midnight_preview");
        _eventDone = true;
        _firstEventCaption = RestaurantTexts.EnText(RestaurantTexts.KeyEvtBlazeBangs);
    }
```

```
}
/// &lt;summary&gt;The stall decision: opens the first preset stove slot
/// AND the counter in one tap (pick WHAT opens). Returns false while
/// the wallet or the rank chain refuses.&lt;/summary&gt;
public bool OpenFirstStall()
{
    if (_stallOpen) return false;
    if (TimeOf("evt_restaurant_hook") &lt; 0) return false;   // the hook owns t0
    if (!_serve.OpenNextStove(0)) return false;
    if (!_serve.OpenStall(0)) return false;
    _stallOpen = true;
    _stallOpenedAt = _t;
    Stamp("evt_stall_opened");
    Stamp("evt_first_decision");
    return true;
}
/// &lt;summary&gt;The quality-audit review beat (T008b): the player requests
/// the audit against the current rank's frozen gate. Pass books the
/// promote stamp triple (the HQ seal + the serving-bell chorus), the
/// event tiers re-scope with the ticket and the economy row settles
/// its promotion (new stoves arrive idle); fail books the inspect
/// triple with the auditor's tears landing on the same tick (the
/// rectification comedy consumes its arm). Returns null before the
/// hook or past the ladder top - the audit is a new-gameplay ticket,
/// never a wall.&lt;/summary&gt;
public RestaurantRankBeat RequestReview()
{
    if (TimeOf("evt_restaurant_hook") &lt; 0) return null;          // the hook owns t0
    if (_rank.GateFor(_rank.Rank) == null) return null;    // the ladder top (R5 has no gate out)
    var beat = _rank.Review(_t, _rank.Rank);
    if (beat.promoted)
    {
        Stamp("evt_promote");
        Stamp("evt_hq_stamp");        // the HQ seal lands
        Stamp("evt_bell_chorus");     // city-wide fireworks + the serving-bell chorus
        _events.SetRank(beat.toRank);  // the event tiers re-scope with the ticket
    }
    else
    {
        Stamp("evt_inspect");
        Stamp("evt_rectify_comedy");   // the auditor cries one bite into a dark dish
        Stamp("evt_auditor_tears");    // the tears land
        if (_rank.RectifyArmed) _rank.ConsumeRectifyArm();   // the comedy beat consumes the arm
    }
    return beat;
}
public int CoinJumpHc =&gt; _coinDone ? _coinJumpHc : 0;
public string FirstEventCaption =&gt; _firstEventCaption;
/// &lt;summary&gt;Offline doubler beat (T010): the claim's x2 landing
/// stamps on the shared stage bus (the offline resolver calls it
```

```
/// right after BookOfflineDouble). Additive: zero calls keep
/// every frozen face byte-identical.&lt;/summary&gt;
public void StampOfflineDouble() =&gt; Stamp("evt_offline_double");
/// &lt;summary&gt;The research-window boost landing (T015b): the completed
/// ad view lands the lightbulb x2 on the current research - the
/// window is the research, the completion tick clears the factor
/// back to neutral. The beat rides the stage stamp list so every
/// stage-level consumer sees it (zero calls keep the frozen face
/// byte-identical).&lt;/summary&gt;
public bool LandBoost(int bp)
{
    if (TimeOf("evt_restaurant_hook") &lt; 0) return false;   // the hook owns t0
    _serve.SetBoostFactorBp(bp);
    Stamp("evt_boost_landed");
    return true;
}
/// &lt;summary&gt;The facility tap (T013c): the build-menu decision lands
/// through the stage so the wallet books the rung on the same tick
/// - the facility ledger and the wallet's facility line reconcile
/// tick-for-tick, a refused pick never half-pays. All three factor
/// faces ride onto the serve engine the moment the build stands;
/// the central kitchen debuts the empire show on the stage bus.&lt;/summary&gt;
public bool TapFacility(int facilityIndex)
{
    if (TimeOf("evt_restaurant_hook") &lt; 0) return false;   // the hook owns t0
    int goldBefore = _serve.GoldHc;
    int spentBefore = _facility.SpentHc;
    if (!_facility.BuildOrUpgrade(facilityIndex, _rank.Rank, goldBefore)) return false;
    _serve.BookFacilitySpend(_facility.SpentHc - spentBefore);   // the wallet pays the same tick
    _serve.SetFacilityFactors(_facility.DiningHallCapBp, _facility.PrivateRoomTicketBp, _facility.CentralKitchen
    Stamp("evt_facility_built");   // the menu closes, the dust settles
    if (facilityIndex == RestaurantFacility.CentralKitchen)
        Stamp("evt_empire_show");   // the R5 empire show debuts on the stage bus
    return true;
}
/// &lt;summary&gt;The review-answer tap (T014b): the boss answers the
/// oldest pending review with the chosen slot. The budget check runs
/// against the live wallet inside the engine (insufficient gold
/// refuses clean - never half-pays), the comp books the wallet line
/// the same tick (the two ledgers reconcile tick-for-tick), the word
/// swing settles through the serving engine, and the stamp lands on
/// the stage bus - the comp also debuts the freebie-drama-king
/// same-tick cameo (the visible comedy of the E03 mirror).&lt;/summary&gt;
public bool TapRespond(RestaurantResponseKind kind)
{
    if (TimeOf("evt_restaurant_hook") &lt; 0) return false;   // the hook owns t0
    int goldBefore = _serve.GoldHc;
    if (!_reviewResp.Respond(_t, kind, goldBefore)) return false;   // clean refuse: zero footprint
    int cost = RestaurantReviewResponse.CostOf(kind);
    if (cost &gt; 0) _serve.BookResponseSpend(cost);   // the wallet pays the same tick
```

```
            Stamp("evt_review_responded");   // the reply lands, the wall hushes
            if (kind == RestaurantResponseKind.CompMeal)
                Stamp("evt_freebie_chain");   // the freebie drama king lands his same-tick cameo
            return true;
        }
        /// &lt;summary&gt;The pending-caption face (T014b): the decision-anchor
        /// toast copy - the pending age on the boss clock.&lt;/summary&gt;
        public string PendingCaption =&gt;
            string.Format(RestaurantTexts.EnText(RestaurantTexts.KeyRespPending), DecisionAnchorAge.ToString("0"));
        /// &lt;summary&gt;The slot-caption face (T014b): the chosen slot's menu
        /// copy (the zh-CN face rides the LocaleService lookup).&lt;/summary&gt;
        public string ResponseCaption(RestaurantResponseKind kind)
        {
            if (kind == RestaurantResponseKind.ReplyThanks)
                return RestaurantTexts.EnText(RestaurantTexts.KeyRespReplyThanks);
            if (kind == RestaurantResponseKind.CompMeal)
                return string.Format(RestaurantTexts.EnText(RestaurantTexts.KeyRespCompMeal), RestaurantReviewResponse.C
            return RestaurantTexts.EnText(RestaurantTexts.KeyRespBossPersonally);
        }
        /// &lt;summary&gt;Opening fingerprint: every stamp, the wallet and the
        /// serve state - byte-identical across identical scripts.&lt;/summary&gt;
        public string OpeningSig()
        {
            var sb = new System.Text.StringBuilder();
            foreach (var s in _stamps) sb.Append(s.id).Append('@').Append(s.t.ToString("0.###")).Append(';');
            sb.Append(_serve.Fingerprint());
            return sb.ToString();
        }
    }
}
// RestaurantTexts.cs
// RestaurantTexts.cs - the diner copy law for Crazy Restaurant (G14-T004b).
// English source strings live here as compiled code (the family source
// law: code strings stay English); the zh-CN localization rides a separate
// data file (Assets/_Game/Configs/Texts_zhCN.json - data, not source).
// Lookup follows the shared LocaleService contract: a missing key falls
// back to the key itself (machine-readable, never a crash), and a zh
// entry missing from the data file falls back to the English source.
// Zero scattered copy: every UI-facing sentence routes through these keys
// - DataSchemaCheck scans the runtime tree for stray CJK and proves the
// coverage law. The diner disclaimer is a permanent resident key and the
// G11 office-worker cameo keeps its reserved seat (S3+ LiveOps face).
using System;
using System.Collections.Generic;
using Sy.Core;
namespace Sy.CrazyRestaurant
{
    public static class RestaurantTexts
    {
        // ---- key constants (the compiled copy surface) ----
```

```
public const string KeyHookCaption = "hook_caption";
public const string KeyDinerDisclaimer = "diner_disclaimer";   // the permanent compliance face
public const string KeyRank1 = "rank_1";
public const string KeyRank2 = "rank_2";
public const string KeyRank3 = "rank_3";
public const string KeyRank4 = "rank_4";
public const string KeyRank5 = "rank_5";
public const string KeyStatGold = "stat_gold";
public const string KeyStatWord = "stat_word";
public const string KeyStatStoves = "stat_stoves";
public const string KeyStatAlbum = "stat_album";
public const string KeyStatDishes = "stat_dishes";
public const string KeyStoveOpen = "stove_open";
public const string KeyStoveSlots = "stove_slots";
public const string KeyServeFirst = "serve_first";
public const string KeyDishReady = "dish_ready";
public const string KeyWordBad = "word_bad";
public const string KeyWordPlain = "word_plain";
public const string KeyWordGood = "word_good";
public const string KeyWordHot = "word_hot";
public const string KeyAuditStreet = "audit_street";
public const string KeyAuditWall = "audit_wall";
public const string KeyAuditDiner = "audit_diner";
public const string KeyAuditEmpire = "audit_empire";
public const string KeyGatePromote = "gate_promote";
public const string KeyGateRectify = "gate_rectify";
public const string KeyRecPlainNoodle = "rec_plain_noodle";
public const string KeyRecBigplateNoodle = "rec_bigplate_noodle";
public const string KeyRecMapoTofuRice = "rec_mapo_tofu_rice";
public const string KeyDarkBanlangenNoodle = "dark_banlangen_noodle";
public const string KeyDarkStrawberryPork = "dark_strawberry_pork";
public const string KeyDarkChildhoodSet = "dark_childhood_set";
public const string KeyDarkSnailCup = "dark_snail_cup";
public const string KeyChefBlaze = "chef_blaze";
public const string KeyChefZencut = "chef_zencut";
public const string KeyChefCashier = "chef_cashier";
public const string KeyChefDishwasher = "chef_dishwasher";
public const string KeyChefSpeedy = "chef_speedy";
public const string KeyChefTaster = "chef_taster";
public const string KeyCustReviewWriter = "cust_review_writer";
public const string KeyCustFreebieActor = "cust_freebie_actor";
public const string KeyCustVlogger = "cust_vlogger";
public const string KeyCustMidnightStranger = "cust_midnight_stranger";
public const string KeyEvtBlazeBangs = "evt_blaze_bangs";
public const string KeyEvtMidnightVisit = "evt_midnight_visit";
public const string KeyEvtReviewEssay = "evt_review_essay";
public const string KeyEvtFreebieActor = "evt_freebie_actor";
public const string KeyEvtDarkFirstBoom = "evt_dark_first_boom";
public const string KeyEvtVloggerVisit = "evt_vlogger_visit";
public const string KeyEvtDarkSecondHit = "evt_dark_second_hit";
```

```
public const string KeyEvtFoodfestChallenge = "evt_foodfest_challenge";
public const string KeyEvtMidnightStranger = "evt_midnight_stranger";
public const string KeyEvtZencutArtwork = "evt_zencut_artwork";
public const string KeyEvtAuditGrand = "evt_audit_grand";
public const string KeyAlbumTitle = "album_title";
public const string KeyAlbumVolumeChef = "album_volume_chef";
public const string KeyAlbumVolumeCustomer = "album_volume_customer";
public const string KeyAlbumVolumeDark = "album_volume_dark";
public const string KeyOfflineSummary = "offline_summary";
public const string KeyOfflineDouble = "offline_double";
public const string KeyOfflinePile = "offline_pile";             // the takeout-mountain line (T010)
public const string KeyOfflineEarned = "offline_earned";         // the closed-door income line (T010)
public const string KeyOfflineSignature = "offline_signature";   // the signature-dish flex line (T010)
public const string KeyShareChallenge = "share_challenge";
public const string KeyDailyReport = "daily_report";
public const string KeyDrawTen = "draw_ten";
public const string KeyNewJourney = "new_journey";
public const string KeyNewJourneyConfirm = "new_journey_confirm";
public const string KeyCarryLabel = "carry_label";
public const string KeyResearchStart = "research_start";
public const string KeyMuseumEntry = "museum_entry";
public const string KeyReviewCollected = "review_collected";
public const string KeyLovedEgg = "loved_egg";
public const string KeyG11OfficeCameo = "g11_office_cameo";
public const string KeyAuditNotice = "audit_notice";
public const string KeyReportHeader = "report_header";           // the menu-style card header (T011a)
public const string KeyReportSignature = "report_signature";     // the signature-dish-of-the-house line (T011a)
public const string KeyReportAlbums = "report_albums";           // the three-volume count line (T011a)
public const string KeyReportWord = "report_word";               // the word-of-mouth line (T011a)
public const string KeyReportFooter = "report_footer";           // the kitchen crew's sign-off (T011a)
public const string KeyReportWallTitle = "report_wall_title";    // the review-literature wall title (T011a)
public const string KeyRespPending = "resp_pending";                 // the decision-anchor toast (T014b)
public const string KeyRespReplyThanks = "resp_reply_thanks";         // the reply-thanks slot label (T014b)
public const string KeyRespCompMeal = "resp_comp_meal";              // the comp-meal slot label (T014b)
public const string KeyRespBossPersonally = "resp_boss_personally";  // the boss-personal slot label (T014b)
// ---- the review-literature genre-signature family (T028): eight
// deadpan registers, each essay a genre-signed review; the labels
// ride the wall line format below ----
public const string KeyReviewEssayNews = "review_essay_news";
public const string KeyReviewEssayDocu = "review_essay_docu";
public const string KeyReviewEssaySports = "review_essay_sports";
public const string KeyReviewEssayWuxia = "review_essay_wuxia";
public const string KeyReviewEssayGatha = "review_essay_gatha";
public const string KeyReviewEssayStele = "review_essay_stele";
public const string KeyReviewEssayAcrostic = "review_essay_acrostic";
public const string KeyReviewEssayFoodcritic = "review_essay_foodcritic";
public const string KeyGenreNews = "genre_name_news";
public const string KeyGenreDocu = "genre_name_docu";
public const string KeyGenreSports = "genre_name_sports";
public const string KeyGenreWuxia = "genre_name_wuxia";
```

```
public const string KeyGenreGatha = "genre_name_gatha";
public const string KeyGenreStele = "genre_name_stele";
public const string KeyGenreAcrostic = "genre_name_acrostic";
public const string KeyGenreFoodcritic = "genre_name_foodcritic";
public const string KeyReportWallEssayLine = "report_wall_essay_line";  // NO.{0} [genre] essay (T028)
/// &lt;summary&gt;English source table (compiled - the source-of-law face).&lt;/summary&gt;
public static readonly Dictionary&lt;string, string&gt; En = new Dictionary&lt;string, string&gt;
{
    { KeyHookCaption, "ONE WOK, ONE DREAM - MY DINER LIFE STARTS NOW" },
    { KeyDinerDisclaimer, "FOR ENTERTAINMENT ONLY - NOT REAL DINING ADVICE" },
    { KeyRank1, "STREET STALL" },
    { KeyRank2, "HOLE-IN-THE-WALL" },
    { KeyRank3, "OLD TOWN DINER" },
    { KeyRank4, "CHAIN BANQUET HALL" },
    { KeyRank5, "GLOBAL FOOD EMPIRE" },
    { KeyStatGold, "GOLD" },
    { KeyStatWord, "WORD" },
    { KeyStatStoves, "STOVES" },
    { KeyStatAlbum, "ALBUM" },
    { KeyStatDishes, "DISHES" },
    { KeyStoveOpen, "OPEN A STOVE" },
    { KeyStoveSlots, "STOVES: {0}/{1}" },
    { KeyServeFirst, "SERVE THE FIRST BOWL" },
    { KeyDishReady, "WOK TO COUNTER - COINS INCOMING" },
    { KeyWordBad, "THE REVIEW WALL FLOODS - SHARP VLOGGER TAKES INCOMING" },
    { KeyWordPlain, "SLOW LUNCH HOURS, BUSINESS AS USUAL" },
    { KeyWordGood, "QUEUES OUT THE DOOR" },
    { KeyWordHot, "THE FOOD SHOW VISITS + BUYOUT OFFERS" },
    { KeyAuditStreet, "STREET QUALITY AUDIT" },
    { KeyAuditWall, "WALL-TO-WALL QUALITY AUDIT" },
    { KeyAuditDiner, "DINER QUALITY AUDIT" },
    { KeyAuditEmpire, "EMPIRE QUALITY AUDIT" },
    { KeyGatePromote, "PROMOTED! {0} UNLOCKED" },
    { KeyGateRectify, "THE AUDITOR CRIED ONE BITE IN - RECTIFY AND RESUBMIT" },
    { KeyRecPlainNoodle, "PLAIN NOODLE SOUP" },
    { KeyRecBigplateNoodle, "BIG-PLATE NOODLE" },
    { KeyRecMapoTofuRice, "MAP TOFU OVER RICE" },
    { KeyDarkBanlangenNoodle, "BANLANGEN INSTANT NOODLE" },
    { KeyDarkStrawberryPork, "STRAWBERRY BRAISED PORK" },
    { KeyDarkChildhoodSet, "CHILDHOOD COMBO SET" },
    { KeyDarkSnailCup, "SNAIL-NOODLE COFFEE" },
    { KeyChefBlaze, "BLAZE (BANGS ON FIRE)" },
    { KeyChefZencut, "ZEN CUT (HALF A BEAT SLOW)" },
    { KeyChefCashier, "CASHIER GIRL (MENTAL MATH QUEEN)" },
    { KeyChefDishwasher, "SLACKER DISHWASHER" },
    { KeyChefSpeedy, "SPEEDY THE RUNNER" },
    { KeyChefTaster, "BRAVE TASTER" },
    { KeyCustReviewWriter, "REVIEW-LITERATURE MASTER" },
    { KeyCustFreebieActor, "FREEBIE DRAMA KING" },
    { KeyCustVlogger, "VLOGGER (THREE LIGHTMEN)" },
```

```
{ KeyCustMidnightStranger, "MIDNIGHT DINER MYSTERY" },
{ KeyEvtBlazeBangs, "THE CHEF'S BANGS CAUGHT FIRE!" },
{ KeyEvtMidnightVisit, "MIDNIGHT: A MYSTERY GUEST SLIDES INTO THE BOOTH" },
{ KeyEvtReviewEssay, "FIRST REVIEW ESSAY: 'GOOD FOOD - BUT THE BOSS LOOKS LIKE MY EX'" },
{ KeyEvtFreebieActor, "THE FREEBIE DRAMA KING SPILLS TOMATO JUICE ON HIMSELF - OSCAR!" },
{ KeyEvtDarkFirstBoom, "FIRST DARK DISH OUT OF THE WOK - THE TASTER'S FACE GOES VIRAL" },
{ KeyEvtVloggerVisit, "THE VLOGGER WALKS IN WITH THREE LIGHTMEN - GOING LIVE!" },
{ KeyEvtDarkSecondHit, "STRAWBERRY BRAISED PORK LANDS - THE LITERATURE MASTER ASKS FOR SECONDS" },
{ KeyEvtFoodfestChallenge, "FOOD-FEST EATING CONTEST - THE HOUSE IS PACKED" },
{ KeyEvtMidnightStranger, "MIDNIGHT MYSTERY: PRICIEST DISH, BIGGEST TIP, FEWEST WORDS" },
{ KeyEvtZencutArtwork, "ZEN PREP PEELS A POTATO INTO A MONA LISA - SLOW BUT GOLD" },
{ KeyEvtAuditGrand, "EMPIRE AUDIT: THE LITERATURE MASTER WAS THE UNDERCOVER ALL ALONG" },
{ KeyAlbumTitle, "DINER ALBUM" },
{ KeyAlbumVolumeChef, "CHEF VOLUME" },
{ KeyAlbumVolumeCustomer, "CUSTOMER VOLUME" },
{ KeyAlbumVolumeDark, "DARK-DISH VOLUME" },
{ KeyOfflineSummary, "WHILE YOU WERE GONE, TAKEOUT ORDERS PILED INTO A MOUNTAIN" },
{ KeyOfflineDouble, "DOUBLE IT?" },
{ KeyOfflinePile, "TAKEOUT MOUNTAIN: {0} ORDERS TAKEN - {1} GOLD PILED UP" },
{ KeyOfflineEarned, "CLOSED-DOOR LEDGER: {0} GOLD BANKED WHILE THE STALL SLEPT" },
{ KeyOfflineSignature, "SIGNATURE OF THE HOUSE: {0}" },
{ KeyShareChallenge, "CAN YOUR DINER OUT-COOK MINE?" },
{ KeyDailyReport, "DAY {0}: {1}" },
{ KeyDrawTen, "TEN-CARD HIRING SPREE - THE CREW RUNS ITSELF" },
{ KeyNewJourney, "START A NEW DINER LIFE" },
{ KeyNewJourneyConfirm, "TAP AGAIN - THE CARRY SEEDS YOUR NEXT STALL" },
{ KeyCarryLabel, "NEXT-BOSS CARRY: {0}" },
{ KeyResearchStart, "RESEARCH: {0}" },
{ KeyMuseumEntry, "THE FAILED DISK ENTERS THE DARK-DISH MUSEUM" },
{ KeyReviewCollected, "REVIEW COLLECTED: {0}" },
{ KeyLovedEgg, "WAIT - SOMEONE ACTUALLY LOVES THIS DISH" },
{ KeyG11OfficeCameo, "A FAMILIAR OFFICE WORKER WALKS IN" },
{ KeyAuditNotice, "QUALITY AUDIT AT DAWN - POLISH THAT WOK" },
{ KeyReportHeader, "SIGNATURE RECIPE CARD" },
{ KeyReportSignature, "SIGNATURE OF THE HOUSE: {0}" },
{ KeyReportAlbums, "ALBUMS: CHEFS {0}/{1} - CUSTOMERS {2}/{3} - DARK {4}/{5}" },
{ KeyReportWord, "WORD OF MOUTH: {0}/100" },
{ KeyReportFooter, "GLORY BELONGS TO THE KITCHEN CREW" },
{ KeyReportWallTitle, "REVIEW LITERATURE GRAND PRIX - {0} COLLECTED" },
{ KeyRespPending, "A REVIEW AWAITS YOUR REPLY - THE BOSS CLOCK READS {0}s" },
{ KeyRespReplyThanks, "REPLY WITH THANKS - WARM, FREE, +1 WORD" },
{ KeyRespCompMeal, "COMP A PLAIN NOODLE - {0} GOLD, +2 WORD, THE DRAMA KING WATCHES" },
{ KeyRespBossPersonally, "BOSS REPLIES IN PERSON - READS THE ROOM, +/-2 WORD" },
{ KeyReviewEssayNews, "BREAKING: the street-corner noodle stall reports yet another 'vanishing noodles' inci
{ KeyReviewEssayDocu, "Few notice where this broth comes from. The camera pushes in slow - three drops of oi
{ KeyReviewEssaySports, "AND HE LIFTS - textbook form off the bowl! But look - the noodle snaps mid-air! A d
{ KeyReviewEssayWuxia, "When the bowl lands, the whole table goes still. Broth clear as a mirror, noodles st
{ KeyReviewEssayGatha, "Noodle is emptiness, emptiness is noodle. Only after the last bite does the diner se
{ KeyReviewEssayStele, "I stopped at this stall and carved this stele: the stall keeps clear broth, its nood
{ KeyReviewEssayAcrostic, "Back again.\nAnother bowl slides in,\nCold by the time it lands.\nKept the receip
```

```
            { KeyReviewEssayFoodcritic, "Ambiance: the folding table holds steady, four stars - the chipped corner faces
            { KeyGenreNews, "NEWS DESK" },
            { KeyGenreDocu, "DOCUMENTARY" },
            { KeyGenreSports, "SPORTS DESK" },
            { KeyGenreWuxia, "WUXIA" },
            { KeyGenreGatha, "GATHA" },
            { KeyGenreStele, "STELE" },
            { KeyGenreAcrostic, "ACROSTIC" },
            { KeyGenreFoodcritic, "FOOD CRITIC" },
            { KeyReportWallEssayLine, "NO.{0} [{1}] {2}" },
        };
        /// &lt;summary&gt;English lookup (missing key falls back to the key itself
        /// - machine-readable, never a crash).&lt;/summary&gt;
        public static string EnText(string key)
            =&gt; En.TryGetValue(key, out var v) &amp;&amp; !string.IsNullOrEmpty(v) ? v : key;
        /// &lt;summary&gt;Parses the zh-CN data file payload into a key-to-text
        /// map (values must be non-empty strings; anything else drops the
        /// entry so the lookup degrades to English, never to garbage).&lt;/summary&gt;
        public static Dictionary&lt;string, string&gt; ParseZh(string json)
        {
            var map = new Dictionary&lt;string, string&gt;();
            if (string.IsNullOrEmpty(json)) return map;
            try
            {
                if (!(MiniJson.Deserialize(json) is Dictionary&lt;string, object&gt; d)) return map;
                foreach (var kv in d)
                    if (!string.IsNullOrEmpty(kv.Key) &amp;&amp; kv.Value is string s &amp;&amp; !string.IsNullOrEmpty(s
                        map[kv.Key] = s;
            }
            catch
            {
                // corrupt data file = empty map = English fallback face
                return new Dictionary&lt;string, string&gt;();
            }
            return map;
        }
        /// &lt;summary&gt;LocaleService-compatible lookup delegate over a parsed
        /// zh map: zh-CN hits read the data file, everything else (and any
        /// miss) reads the English source. Wire-ready for the T005 UI face.&lt;/summary&gt;
        public static Func&lt;string, string, string&gt; LookupWithZh(Dictionary&lt;string, string&gt; zh)
        {
            return (lang, key) =&gt;
            {
                if (lang == "zh-CN" &amp;&amp; zh != null &amp;&amp; zh.TryGetValue(key, out var v) &amp;&amp; !string.I
                    return v;
                return EnText(key);
            };
        }
    }
}
```